Programa de Engenharia Elétrica **Curso**: Mestrado Acadêmico

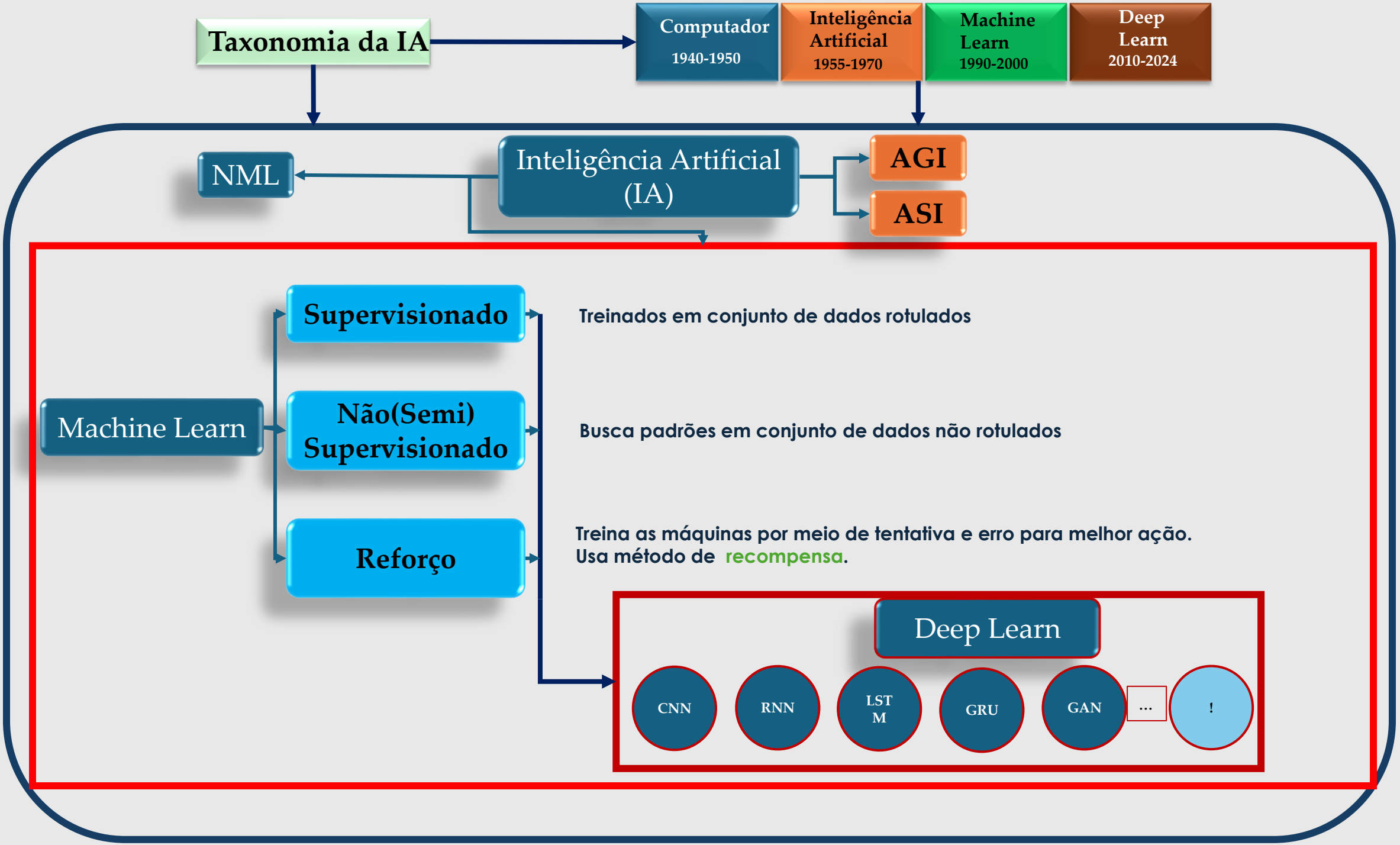
Programa de Pós-graduação em Engenharia Elétrica (PGEE)
Instituto de Ciência e Tecnologia Sorocaba (ICTS).

Nome 1: Levi Rodrigues da Silva PGE240214

Nome 2: Grazielle Cossa

Disciplina: Deep learning for engineering
Professor: Dr. Ivan Aritz Aldaya Garde

Projeto Final 1(3) - Orientação sintetizada/Detalhada para iniciantes**.**


Taxonomia da IA
Computador
1940-1950
Inteligência Artificial
1955-1970
Machine Learn
1990-2000
Deep Learn
2010-2024
NML
Inteligência Artificial (IA)
AGI
ASI
Machine Learn
Supervisionado
Não(Semi) Supervisionado
Reforço
Treinados em conjunto de dados rotulados
Busca padrões em conjunto de dados não rotulados
Treina as máquinas por meio de tentativa e erro para melhor ação.
Usa método de recompensa.
Deep Learn
CNN
RNN
LSTM
GRU
GAN
...
!

IA: Essas máquinas extraem insights e padrões a partir das informações disponíveis em sua memória limitada. É importante saber que a memória dessas máquinas é restrita a um conjunto específico de dados e não suporta aprendizagem duradora como dos humanos.
Inteligência artificial (IA) é um termo abrangente para diferentes estratégias e técnicas usadas para tornar as máquinas mais parecidas com seres humanos. Realiza tarefa complexa com eficiência.
Dentro do campo da IA, conseguimos categorizar os tipos de IA em dois critérios gerais:

- Nível de semelhança com a mente humana, que considera quão próxima a IA está de se assemelhar a um ser humano;
- Critério de funcionalidade e técnica, que avalia a quantidade de tarefas e complexidade de cada sistema.

A **IA** costuma-se dividir em 3 segmentos:

1.Inteligência Artificial Limitada (ANI): programadas para armazenar volume expressivo de dados e executar tarefa (até 2024 tudo que existe de IA).
2.Inteligência artificial geral (AGI): considerada de "nível humano", é capaz de aprender e reagir a estímulos (**conceito** em desenvolvimento)
3.Superinteligência (ASI): ainda em estudo, espera-se ser capaz de superar a inteligência humana, tomar as próprias decisões e executar tarefas que nem os humanos são capazes de fazer (conceito **imaginado**).
ANI=Artificial Narrow Intelligence, **AGI**=Artificial General Intelligence e **ASI**= Artificial Super Intelligence

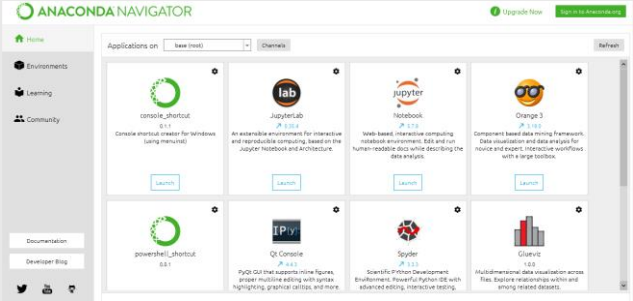
NML – No machine Learn – Todos os conceitos e trabalhos realizados desde sua origem, com a definição do neurônio Perceptron, Adelaine, etc e todos os trabalhos de software e hardware trabalhados que estejam fora do conceito de Machine Learning.

Machine Learning : é um subcampo da Inteligência Artificial (e **Deep Learning** é um subcampo de Machine Learning), que é amplamente definido como a capacidade de uma máquina de imitar o comportamento humano inteligente. Os sistemas de Inteligência Artificial são usados para realizar tarefas complexas de maneira semelhante à forma como os humanos resolvem problemas.
O campo de estudo que dá aos computadores a capacidade de "aprender sem serem explicitamente programados".

A função de um sistema de Aprendizado de Máquina pode ser **descritiva**, o que significa que o sistema usa os dados para explicar o que aconteceu; **preditiva**, significando que o sistema usa os dados para prever o que acontecerá; ou **prescritiva**, o que significa que o sistema usará os dados para fazer sugestões sobre quais ações tomar.
O campo da IA abrange uma variedade de métodos usados para resolver diversos problemas.
Esses métodos incluem algoritmos genéticos, redes neurais, aprendizado profundo, algoritmos de busca, sistemas baseados em regras e o próprio machine learning.

Trabalhar e programar Machine Learn

Ambiente de trabalho para IA: Existem alguns ambientes para o desenvolvimento dos programas criados para a Inteligência Artificial. Alguns deles trabalham no sistema prompt e outros no sistema interface gráfica (notebook) e as vezes nos dois sistemas de comunicação.
Os 3 principais ambientes para o desenvolvimento de programas IA estão listadas abaixo:



Anaconda

O **Anaconda** é um ambiente de trabalho muito usado que é uma distribuição das linguagens de programação **Python e R** para computação científica, que visa simplificar o gerenciamento e implantação de pacotes, controlando o ambiente. A distribuição inclui pacotes de ciência de dados adequados para Windows, Linux e macOS.
Este ambiente utilizam bibliotecas de SW embarcadas, Application Programming Interface (API) com inúmeras ferramentas framework para usos específicos e não cria conflitos de revisões entre outras vantagens.
Conhecido como notebook, pela forma de uso e desenvolvimento do trabalho mas pode-se programar em sistema de prompt.
Existe o pacote grátis e individual e os pacotes pagos e empresariais:
https://www.anaconda.com/

Google Colab

Colab: Ao invés de usarmos o Anaconda, podemos usar outra ferramenta para trabalharmos no ambiente com Python, chamado Colab. Esta ferramenta da Google, simula virtualmente um computador **CPU** e até com o **GPU**. Nesta máquina virtual, que normalmente é mais poderosa que as nossas máquinas. Para acessar https://colab.research.google.com/
Existe uma versão "pro" paga, que garante mais recursos ou máquinas virtuais melhores aos programadores.
Muito usado no mundo acadêmico.
Atualmente está embarcado também a IA com nome de **Gemini** que auxilia nas programações do Python informando erros de sintaxe e sugerindo solução para diversos problemas.

Jupyter

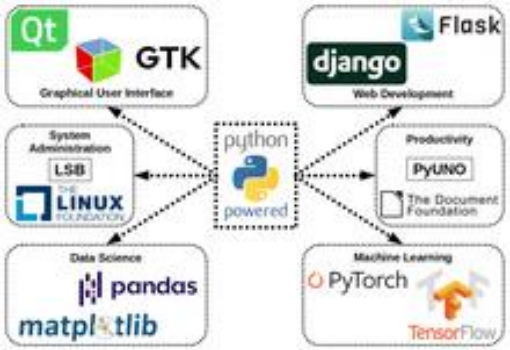
O JupyterLab: é ambiente de desenvolvimento interativo baseado na web para notebooks, código e dados. Sua interface flexível permite que os usuários configurem e organizem fluxos de trabalho em ciência de dados, computação científica, jornalismo computacional e aprendizado de máquina. Um design modular convida extensões para expandir e enriquecer a funcionalidade Project Jupyter | Home .

Trabalhar e programar Machine Learn

Software: Os softwares já usados no ambiente para desenvolvimento de trabalho do IA, temos alguns nomes como:
Python, R, Julia, Scala, Java, C++, Javascript, Lisp, Haskell e Mojo. O mais utilizado em 2024 por desenvolvedores, cientistas e engenheiros é o **Python**.

Python

- **Notebook Comandos**
- **Bibliotecas**
- **Framework**



wikipedia

Na ciência da computação, **Biblioteca** é uma coleção de subprogramas utilizados no desenvolvimento de software.
Eles contém funções e dados auxiliares que podem ser usados em programas independentes, o que permite o compartilhamento e a alteração de código-fonte e dados de forma modular.
Framework de software compreende de um conjunto de classes implementadas em uma linguagem de programação específica, usadas para auxiliar o desenvolvimento de software.
O conceito não é de um software executável, mas um modelo de dados para um domínio.
Tanto frameworks quanto bibliotecas são códigos escritos por outras pessoas que é usado para resolver problemas comuns e geralmente públicos (código aberto).
Ao usar uma biblioteca, você está no controle do fluxo da aplicação. Você seleciona quando e onde chamar a biblioteca. Quando você usa um framework, é ele quem está ao encargo do fluxo. Ele fornece alguns lugares onde você pode encaixar o seu código, mas é ele quem chama o código que você encaixou quando for necessário.
Analogicamente falando, o framework é o esqueleto ou planta de uma casa que você constrói, que após construída, não tem como alterar.
Você pode instalar dentro da casa, móveis por exemplo, que são as bibliotecas. Estes móveis você pode alterar quando desejar.
A casa você pode reformar mas jamais alterar o esqueleto principal dela, a não ser que faça a demolição total e usa outra planta e construção.

Os principais e conhecidos framework e bibliotecas estão listadas mas para conhecer elas deve entrar em suas documentações, são: **Pytorch** (gráfico computacional), **Scikit- Learn (**Machine Learn), **TensorFlow (**Deep Learn**), matplotlib** (gráficos), **Keras (**Redes Neurais), **LangChain** (modelos para linguagem grande (LLM) e integra com inúmeras ferramentas), **Hugging Face (**Machine Learn), **OpenNN** (Redes Neurais), **OpenAi** (GPT e IA), **PyBrain** (Machine Learn), **IBM Watson (**Machine Learn e IA), **CNTK** (Deep Learn), **DL4L** (Deep Learn), **Theano** (Computação numérica), **MXNet** (Pesquisa e produção de IA), **Caffe** (Deep Learn), **XGBoost** (Aumento de gradiente), **Llhama** (Tipo GPT e IA) e **JAX** (IA).

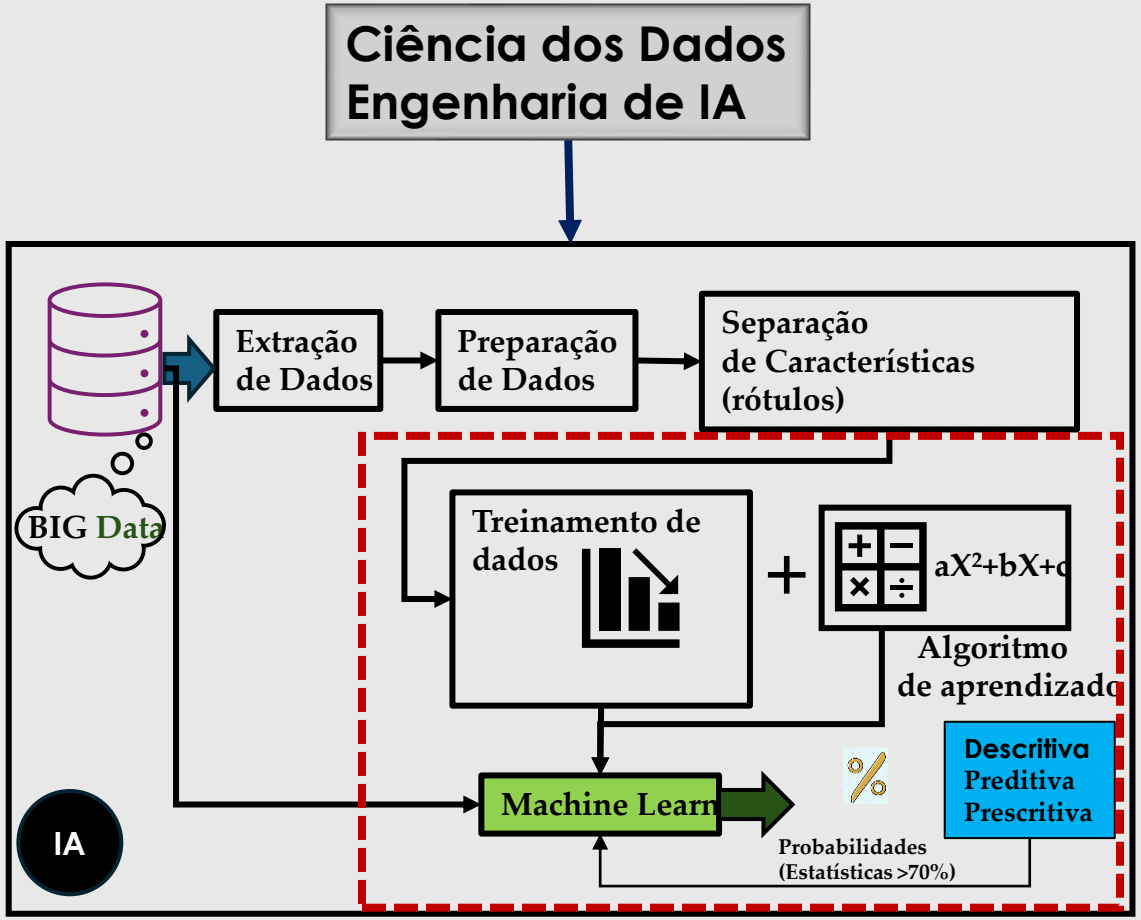
1950 (Alan Turing): **Pode as máquinas pensar?**

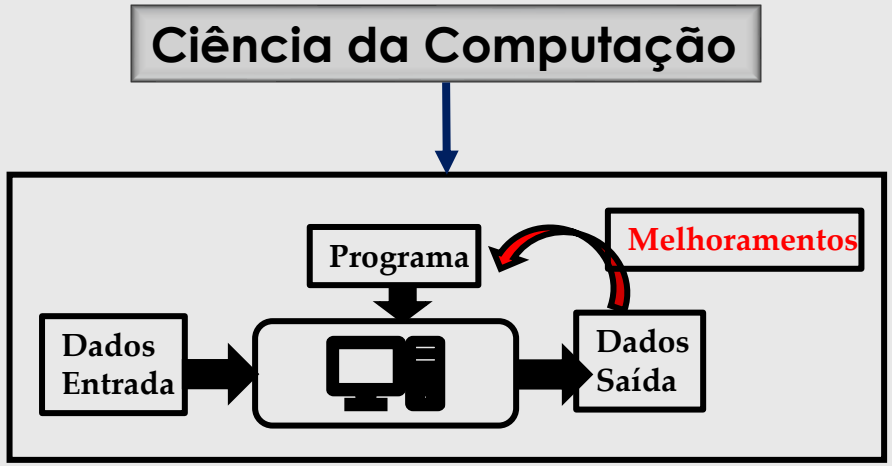
"Um computador era capaz de "pensar" se seus resultados fossem tão convincentes que uma pessoa que interagisse com ele não pudesse distinguir suas respostas das de um ser humano real", ficando conhecido como "**Teste de Turin**"

1956: **Nascimento da Inteligência Artificial (IA) – Dartmouth Workshop de verão.**

Em 1955, John MacCarthy, então um jovem professor assistente de matemática no Dartmouth College, decidiu organizar um grupo de matemáticos e cientistas para esclarecer e desenvolver ideias sobre máquinas pensantes.

Em 2 de setembro de 1955, o projeto de pesquisa foi formalmente proposto por McCarthy, Marvin Minsky, Nathaniel Rochester e Claude Shannon e logo após creditada a eles a famosa frase que foi introduzida desde então, o termo "**inteligência artificial**".





Observação do autor: Antes era bem separado as funções e desenvolvimentos de programas.

Atualmente (2024) os trabalhos envolveram todas as profissões e fica até difícil distinguir esta separação.

A dedicação exclusiva a um determinado nicho dos trabalhos que distinguem as diversas profissões novas catalogadas e até as não catalogadas e que irão surgir.

A **filosofia** entra fortemente neste campo, com as discussões, controvérsias e medos que acompanha esta nova tendência.

O homem que desenvolve pode levar discriminação, preconceito, racismo, tendências religiosas ou de ateísmo, militarismo, Deep Fake, pseudociência, etc.

Machine Learn

Machine Learn (ML)

Não Supervisionado

Clustering — Segmentar por cliente, Mercado alvo e Sistema recomendado.

Reduction — Visualização Big Data, compressão significativa, Descoberta de estrutura e Elicitação de recurso.

Supervisionado

Classificação — Detecção de fraudes, Classificação de imagens, Retenção de clientes e Diagnósticos

Regressão — Novas percepções, Otimização de processos, Predição e Previsão (**Estatística – Regressão Linear** com equação do primeiro grau, **Logística** – Binária (0 ou 1) e **Regressão Polinomial** de segundo ou grau maior.)

Reforço

Decisão tempo real
Game com IA
Aprendizado de tarefas
Aquisição de habilidades por interação
Navegação com robô

Recompensa (+/-): Realizada por cada ação completada.

+1 -1

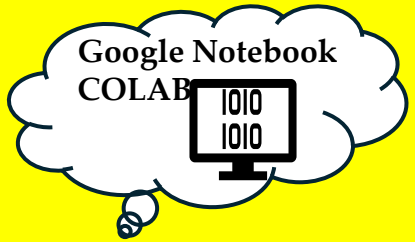
Processamento de Linguagem Natural — Siri, Alex, ChatBox, etc

Redes Neurais — As redes neurais artificiais são modeladas a partir do funcionamento do cérebro humano, no qual milhares ou milhões de nós de processamento são interconectados e organizados em camadas.

Deep Learn — As redes de aprendizado profundo são redes neurais com muitas camadas. Veículos autônomos, chatbox, diagnósticos médicos, algoritmos de recomendação, análise de imagens e detecção de objetos, detecção de fraude, etc.

Ambientes de trabalho

Jupiter Notebook/Anaconda)



Poderoso e Interativo
Permite relatórios completos
Download e compartilhável
(.ipynb e py)

Simula na nuvem o HW necessário (CPU e GPU): Grátis limitado ou pago com melhor performance.
Usa IA para auxiliar o usuário.

SW – Interpretado (ML)

Python

Commands

Operators

Variables

Data: bool, int, float, complex Bytes, strings, bytearray, list [], tuple(), Range, dict {} and set {}, dataframe.

Loops: for i in X / While loop

Function: def / return

Comprehension - Multiple commands on the same line

Files: reading/writing/copying

Classes and Objects

Machine Learn (ML) Supervisionado

Regressão (Qtd)/Classificação (classes)

Linear - uma linha reta como: y = ax+b
Polymonal – curvas com X^2, X^3, ...X^n
Logística – Classificação binária (0,1)
Erros ou Cost:
Mean Absolute Error (MAE)
Mean Squared Error (MSE)
Root Mean Squared Error (RMSE)
Relative Absolute Error (RAE)
Relative Squared Error (RSE)
R-Squared (R2)
OBSERVAÇÃO: Tudo aplicado em Gradiente Descendente (GD).

Função de custo = busca pelo valor mínimo da curva = GD (Θ - derivada parcial).
Taxa de aprendizado (passos) – (0 a 1) – muito pequeno (α))

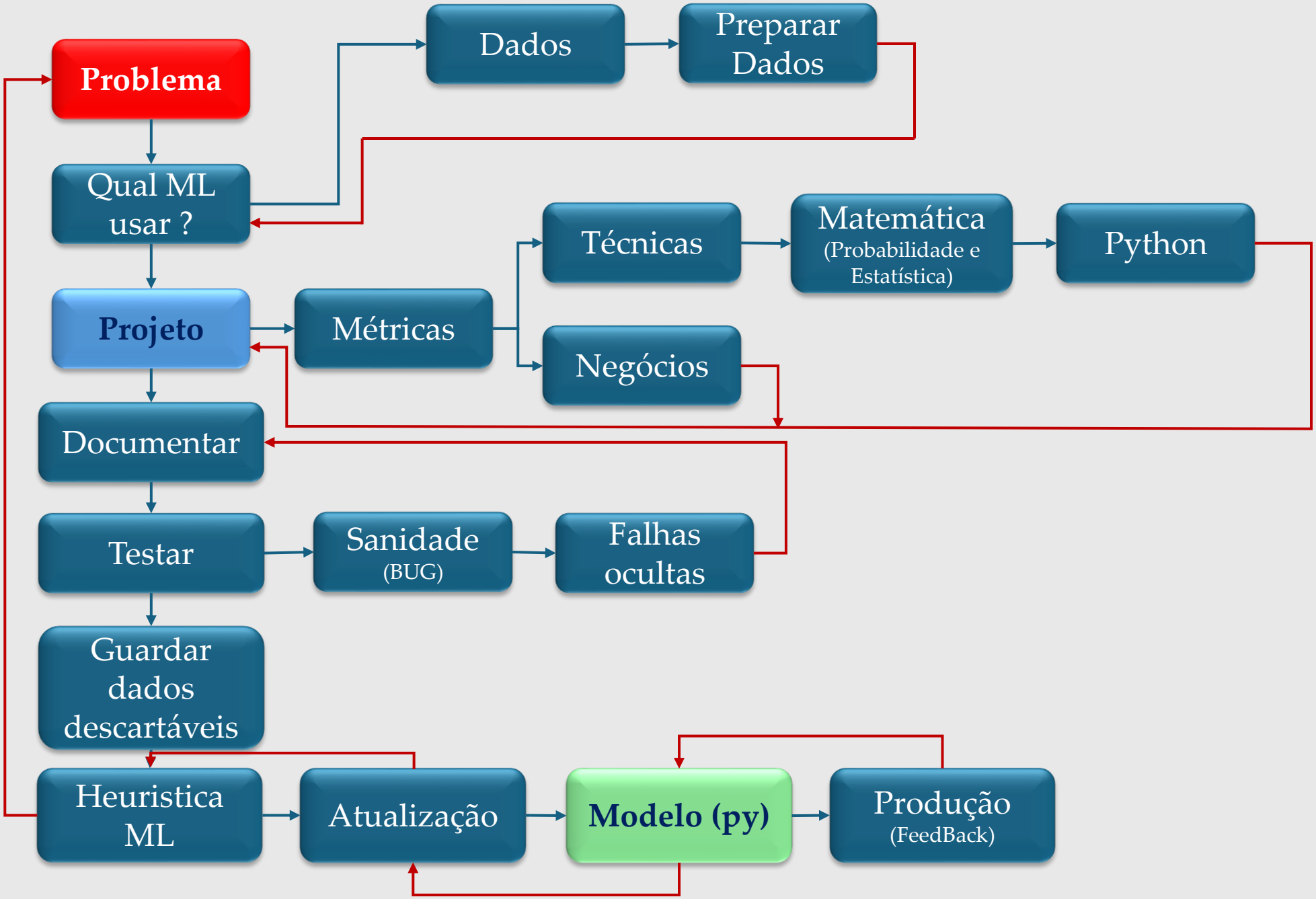
Bibliotecas Python:

Matplotlib (plt):
Criar gráficos
Subplots - linha, coluna
Multiplas figura
Texto desenhado
Plot 3D, histogramas, etc
Imagens
Numpy (np): objeto array contínuo e rápido
Algebra linear, Fourier e matrizes
Números randômetros
Matemática e estatística

Bibliotecas Python:

Pandas (pd):Manipulação de dados e análises
Series: linhas de dados (vertical ou horizontal)
Dataframe: Conjunto de series (tabelas)
Panel: dictionary de Dataframes
Possuí Matplotlib
Save (jason, html, excell-csv) e load

Organograma de Trabalho com Machine Learning



Glossário

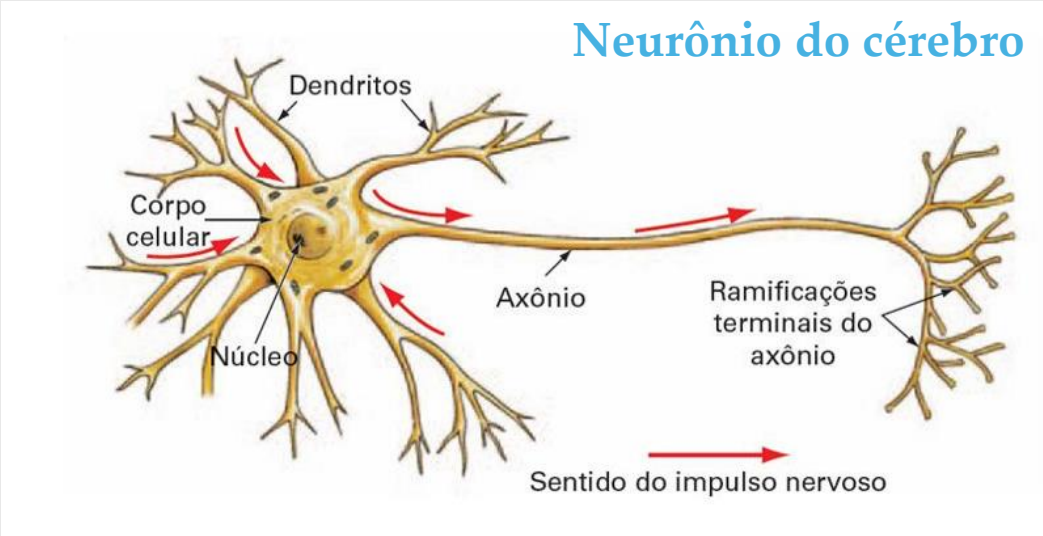
Os termos a seguir aparecerão repetidamente em projetos de Machine Learning.
A definição deve ser buscada na internet.

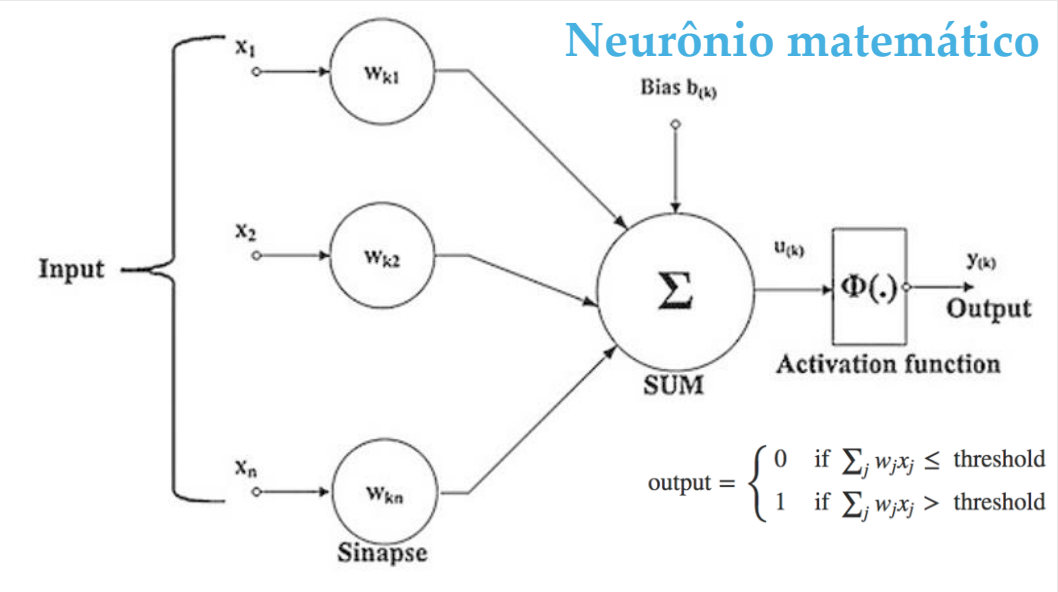
Glossário: Instância, Rótulo, Classe, Feature(Recurso) Denso ou Esparso, Dados categóricos, Dados numéricos, Modelo, Baseline, Métrica, Função Objetivo, AUC (Área da curva Roc), Acurácia, Precisão média, DataFrame, Função de ativação, Backpropagation, Batch, Batch Normalization, Stochastic Gradient Descent SGD), Overfitting, Bagging, Boosting, Bag of Words, Word Embedding, Inteligência Artificial, Recall, Precisão, Bias, Classificação Binária, BLEU (Bilingual Evaluation Understudy (-0 a 1)), Normalização (-1 a 1), Entropia Cruzada, Matriz de Confusão, Validação, Estacionaridade, Convergência, Desbalanceamento de Classe, Threshold (limite), CheckPoint, Metadados, Heurística, Delta, Gradiente, Derivadas Parciais, vetor, Otimizadores, Underfitting, vanishing gradient, Heuristica, agente, ação, Potencial de ativação, Função de ativação, Descritiva,Preditiva, Prescritiva.

Perceptron e Adaline : Primeiro Inverno (1960 a 1970) – Neurônio resolve apenas portas AND, NOR e OR mas não OR Exclusivo. O não progresso da IA devido a vários fatores, entre eles o tecnológico, fez parar o investimento financeiro e a pesquisa diminuiu.
O **Adaline** (widrow e Hoff) é o melhoramento do perceptron que era função degrau e Adaptive Linear Element (Adaline) função ativação linear, acrescentou mais um bloco, minimizando os erros (cost) e o ajuste dos erros é realizado através do gradiente descendente (encontrar o mínimo). O gradiente dá direção, para saber o quanto será o passo a seguir e aplica a **taxa de aprendizagem**.
Segundo Inverno: Na década de 1980 com a evolução da tecnologia e novas descobertas de neurônios para serem ativados com funções polinomiais e que resolve a porta OR Exclusivo, com Rumelhart, Hilton e Willians introduzindo **Backpropagation** em 1986, novos investimentos em massa surgiram mas os efeitos não foram suficientes, parando novamente o impulso da IA.
Finalmente o Verão: Em 2010 com a evolução do Machine Learn e chegando ao Deep Learn, os resultados começaram a aparecer no dia a dia das pessoas e então a IA tomou o seu impulso de crescimento, que agora não tem mais volta, todas as profissões devem conhecer pelo menos o que significa a IA e qual impacto trará na sua profissão.
OBS.: Por respeito aos pioneiros da criação de um neurônio matemático, qualquer modelo de neurônio com suas características diferentes, costumamos chamar genericamente de "Perceptron".

Estudo Aprofundado

Machine Learn Supervisionado

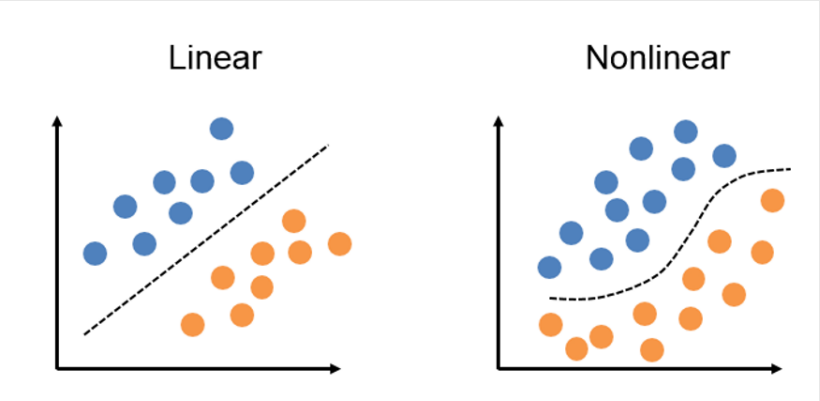




Dois neurônios matemáticos são básicos: **Perceptron** **e** **sigmoide**. Existem muitos outros.
Eles são o algoritmo básicos para aprendizagem da rede conhecido como **descida estocástica do gradiente (SGD)**.
Existem inúmeras arquiteturas (exemplo CNN) que são programadas com **Python** e Google **TensorFlow**.
O Perceptron é classificador linear binário (0 ou 1). As funções lógicas da matemática é **AND-OR-NAND**. O **NAND** são circuitos universais para a computação mas o problema de não poder classificar o **XOR** foi um dos motivos que levou a IA em um período chamado de Inverno da IA, onde não se desenvolveu esta tecnologia.
Com novas ferramentas e recursos, também a criação do neurônio sigmoide, este problema foi resolvido (XOR) e estamos com IA em pleno desenvolvimento e sem volta ao inverno de IA.

X1, X2, ...Xn – Entradas.
Dendritos e axônios = **Pesos sináptico**. Formam matrizes (vetor) de **somas ponderadas** {Σ}.
No neurônio biológico, os sinais são enviados em pulsos e alguns componentes dos neurônios biológicos, a exemplo do axônio, funcionam como filtros de frequência.
Modelo do neurônio matemático também pode incluir uma **polarização** ou **bias de entrada, responsável pelo thresholds** (limiar) de ativação.
Este bias **{b}** especifica qual será o patamar apropriado para que o resultado produzido pelo combinador linear **{Σ}** possa gerar um valor de disparo de ativação.
Potencial de ativação **{u}:** É o resultado obtido pela diferença do valor produzido entre o combinador linear e o limiar de ativação. Se o valor for positivo, ou seja, se $u \geq 0$ então o neurônio produz um potencial excitatório; caso contrário, o potencial será inibitório.
Função de ativação {g}: Seu objetivo é limitar a saída de um neurônio em um intervalo de valores.
Sinal de saída {y}: É o valor final de saída podendo ser usado como entrada de outros neurônios que estão sequencialmente interligados.

Funções lógicas da computação

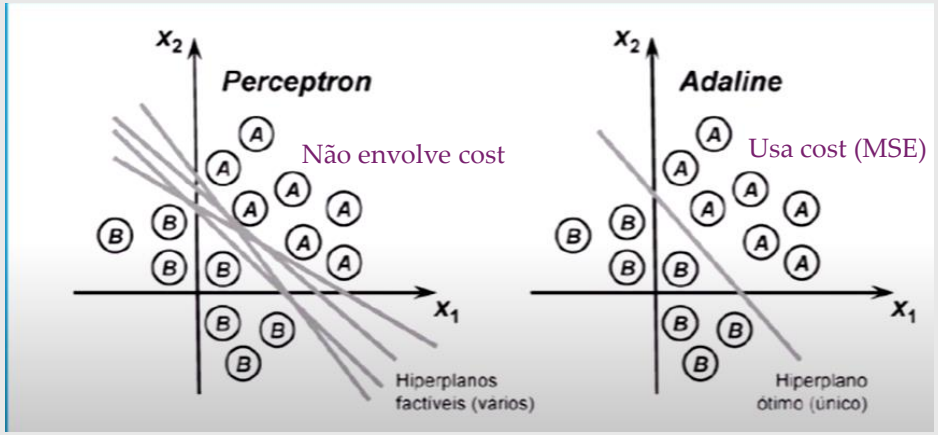


Perceptron Função linear **Função não linear**

Uma rede neural é um mecanismo de aprendizado de máquina (Machine Learning) muito poderoso que imita basicamente como um cérebro humano aprende e usa a computação os valores mais básicos dela que são as lógicas Booleanas com as funções AND, OR e NOR. Um único neurônio matemático chamado de Perceptron (início de tudo) realiza a regressão linear mas não consegue representar o XOR.

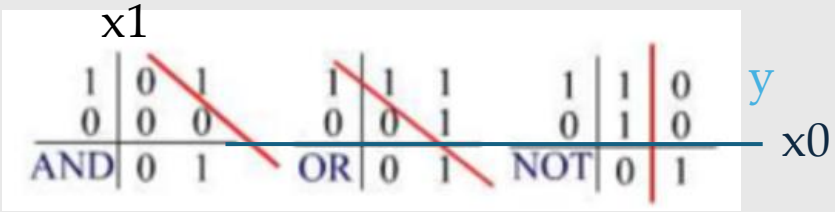
Perceptron (Rosemblatt - 1958)

Diferença do Perceptron x Adaline

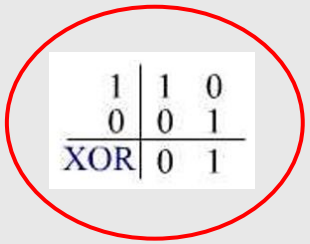


Encontrou uma reta de separação correta, ela para e pode ser uma reta, boa ou não tão boa de separação.

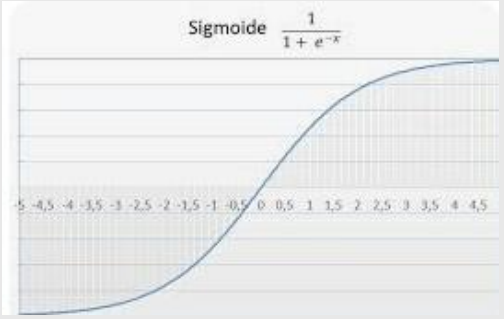
Encontra a única e melhor reta de separação.



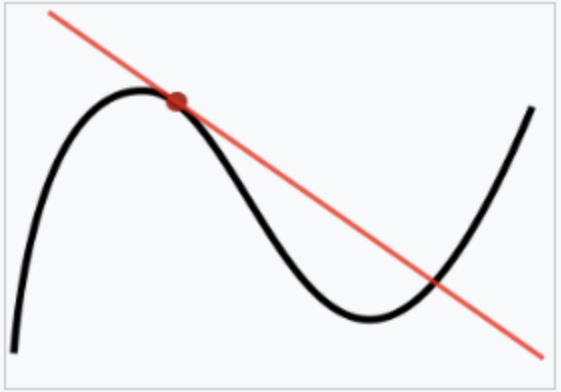
Para resolver o problema da porta XOR e resolver Funções não lineares, foi criado outros modelos de neurônios, como sigmoide, e isso garantiu surgir redes neurais artificiais.



Função lógica que não faz no Perceptron e modelos novos surgiram como o sigmoide



Perceba que o valor vai de 0 a 1 de y não importando os valores de x e se derivar os valores menores que -5 ou maiores que +5 tendem a saturar os resultados.
Por isso surgiram outros modelos de neurônios mas não existe um que seja usável em 100% das programações, apesar que o **ReLu** seja o mais bem aplicado.



O gráfico de uma função, desenhadas em preto, e uma linha tangente a essa função, elaborado em vermelho. A inclinação da linha tangente é igual a derivada da função no ponto marcado.

Wikipedia

"Dizemos que Derivada é a taxa de variação de uma função y = f(x) em relação à x, dada pela relação Δx / Δy. Considerando uma função y = f(x), a sua derivada no ponto x = x0 corresponde à tangente do ângulo formado pela intersecção entre a reta e a curva da função y = f(x), isto é, o coeficiente angular da reta tangente à curva" (Introdução ao Estudo das Derivadas - Brasil Escola (uol.com.br)).

A esta derivada chamada de **ordinária** é representada como:

f(x)=y **f'(x)=**df(x)/d(x) - Isso apenas em um plano de mudança, o plano **x**.

Caso tenhamos outros planos, como por exemplo os planos tridimensionais x,y,z, isso muda.

f(x,y,z)=y f'(x,y,z) = df(x,y,z)/d(x) ou df(x,y,z) / d(y) ou df(x,y,z) / d(z) , ou seja, com 3 planos diferentes teremos não somente aplicar o conceito da derivada mas saber em **qual plano estamos aplicando a derivada.**
A este conceito chamamos de derivada parcial e usamos um símbolo para indicar esta derivada (∂).

No cálculo, nós temos um operador chamado **Nabla** ou Del que trata destas derivadas parciais totais e simbolizado por um triângulo invertido e vetorial ($\vec{\nabla}$).

Quando a entrada da função é escalar e a saída vetorial, chamamos de **Gradiente**.
Se a entrada é vetorial e a saída escalar, chamamos de **Divergente**.
Se for a matriz vetor de uma função vetorial gera uma nova função vetorial chamado de **Rotacional**.

$$\vec{\nabla} \times \vec{F} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ F_1 & F_2 & F_3 \end{vmatrix}$$

$$\vec{\nabla} f = i\frac{\partial f}{\partial x} + \vec{j}\frac{\partial f}{\partial y} + \vec{k}\frac{\partial f}{\partial z}(Gradiente),$$

$$\vec{\nabla} \cdot \vec{F} = \frac{\partial F_1}{\partial x} + \frac{\partial F_2}{\partial y} + \frac{\partial F_3}{\partial z}(Divergente),$$

$$\vec{\nabla} \times \vec{F} = \vec{i}\left(\frac{\partial F_3}{\partial y} - \frac{\partial F_2}{\partial z}\right) + \vec{j}\left(\frac{\partial F_1}{\partial z} - \frac{\partial F_3}{\partial x}\right) + \vec{k}\left(\frac{\partial F_2}{\partial x} - \frac{\partial F_1}{\partial y}\right)(Rotacional).$$

Conclusão: Na **IA** chamamos estas derivadas parciais de **Gradiente** por causa da definição em cálculos matemáticos e esta taxa de variação como a velocidade de aprendizado, que nos coeficientes angulares da tangente na relação trigonométrica, varia de 0 a 1.
Portanto em todas as funções usadas usamos a derivada da função para verificar o quão próximo os valores estão do mínimo, que chamamos de **descendente,** tendo estas variações tão pequenas quanto o conceito do limite pode nos proporcionar.

Ativações com diferentes neurônios

As diversas funções ou logística e suas derivadas usadas são dadas pela fórmulas e gráficos abaixo:

$\sigma(x)=1/(1+e^{-x})$

$\sigma'(x)=\sigma(x).(1-\sigma(x))$

$\tanh(x)=2\sigma(2x)-1$

$\tanh'(x)=1-\tanh^2(x)$

$ReLU(x)=\max\{0,x\}$

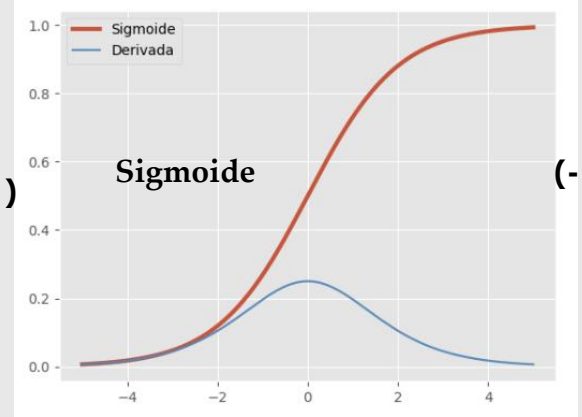
$ReLU'(x)=\begin{cases}1, & \text{se } x\geq 0\\ 0, & \text{c.c.}\end{cases}$

$LeakyReLU(x,a)=\max\{ax,x\}$

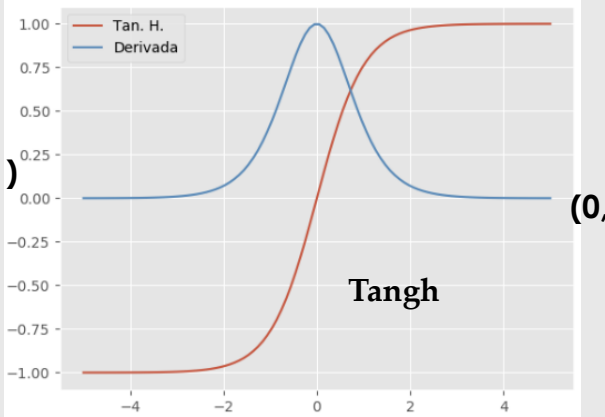
$LeakyReLU'(x,a)=\begin{cases}1, & \text{se } x\geq 0\\ a, & \text{c.c.}\end{cases}$

Suas curvas:

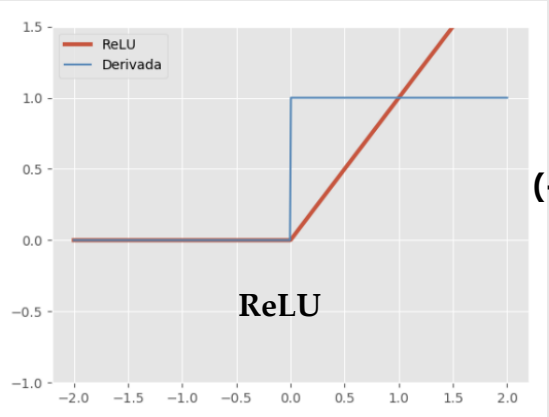
(0,1)



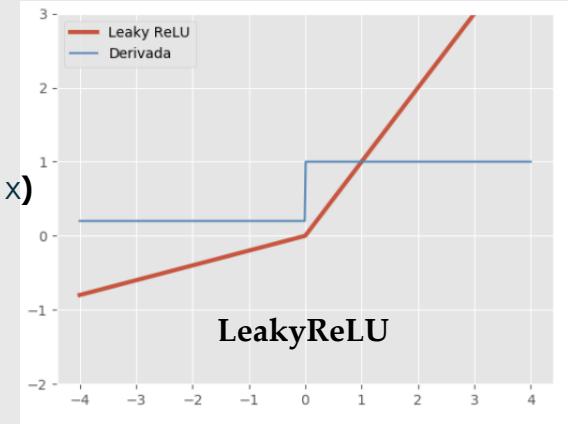
(-1,1)



(0,x)

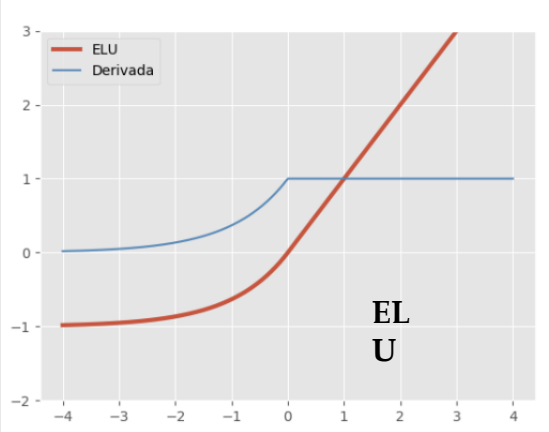


(-1, x)

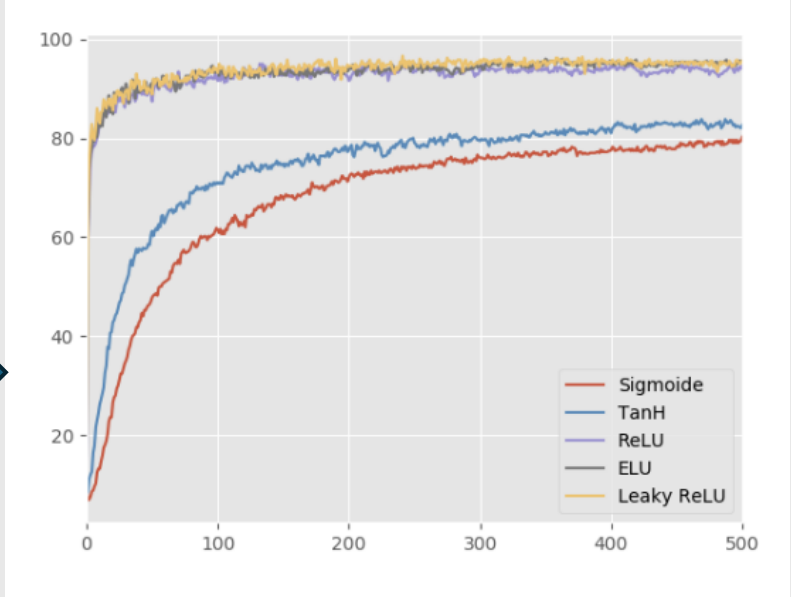


$ELU(x,a)=\begin{cases}x, & x\geq 0\\ a(e^x-1) & \text{c.c.}\end{cases}$

$ELU'(x,a)=\begin{cases}1, & x\geq 0\\ ELU(x,a)+a, & \text{se c.c.}\end{cases}$



Desenvolvendo uma mesma programação Python, usando os diferentes ativadores, temos os resultados gráficos.
Conclui-se que o ReLu e LeakyReLu traz um grande ganho e por isso é tão utilizado nas redes.
Porém os demais ativadores podem ter sua utilidade em determinada aplicação típica.
Existem outros modelos de neurônios.

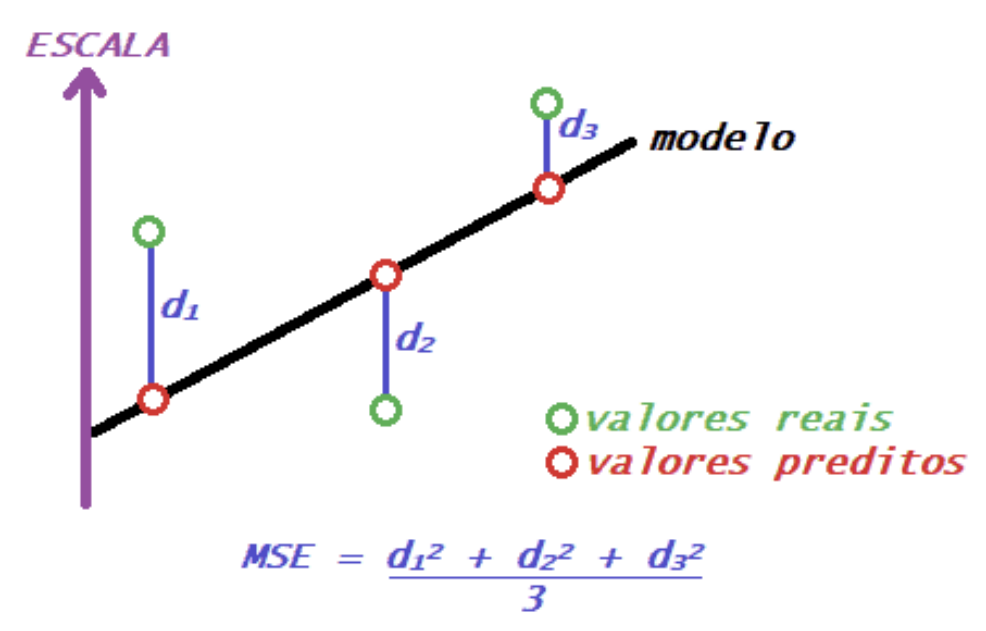


COST na Regressão Logística

Necessário ter uma **métrica** para dizer se ela é boa ou ruim após aplicado a regressão logística. Um dos métodos é usar os **erros**, que podem ter o nome de **cost**. Alguns deles são:

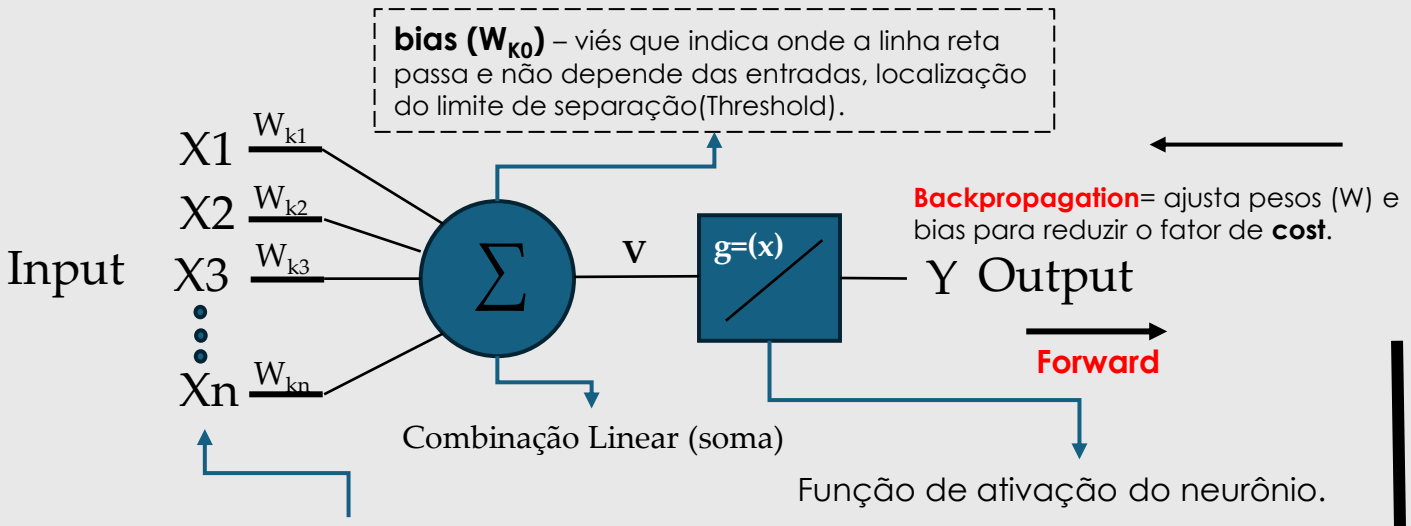
- Mean Absolute Error (MAE)/ Mean Squared Error (MSE)/Root MSE (RMSE)/Relative Absolute Error (RAE)/Relative Square Erros (RSE)/R-Squared (R^2)

A Regressão Logística é a função que representa a reta inclinada preta, no caso Regressão Linear. O d1, d2 e d3 são os erros ou cost entre o valor real e o valor predito.
MSE é levar cada um deles ao quadrado e dividir pela média deles.



Exemplo de erro (cost) (MSE)

Modelo de um Neurônio com todos os conceitos definidos



synaptic weights: são números, positivos ou negativos, que indicam a força ou eficácia do acoplamento sináptico entre um neurônio que envia o sinal e o neurônio que o recebe

Forward – Inicia-se com valores aleatórios dos pesos Wkn que multiplica os valores de entrada Xn.
O bias é aplicado em cada entrada e a somatória total apresenta na saída V.
O potencial V passa pela função de ativação g=(x) e fica g=(V), que traz a resposta para Y informando se ele está ativado (1) ou não (0).
Esta ativação tem um cost (erro) que é calculado. Se não for o menor erro, o Backpropagation é iniciado.
Backpropagation é um método que nos permite atualizar os pesos de uma rede neural multicamadas. Usa-se a regra da cadeia com o gradiente descendente (derivada de função de uma função) para otimizar a função da rede.

$$v = \left(\sum_{I=1}^{n} w_i X_i\right)$$

$$\Theta$$

$$Y = g(v)$$

Gradient descendente=(Θ) $\vec{\nabla}$ **f** = Usa Operador Nabla para derivadas parciais onde entra uma função escalar e obter uma função vetorial (matriz).

$$\vec{\nabla} = \vec{i}\frac{\partial}{\partial x} + \vec{j}\frac{\partial}{\partial y} + \vec{k}\frac{\partial}{\partial z}$$

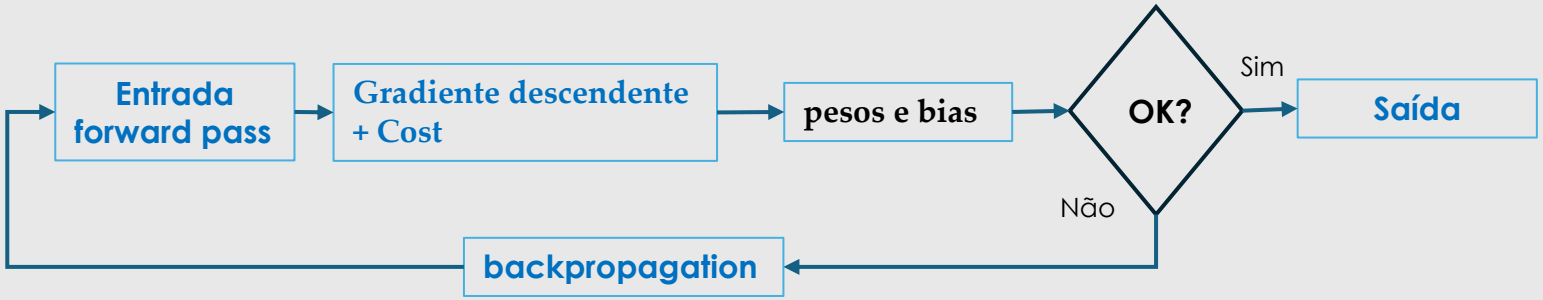
Principais Arquiteturas de Rede Neurais:

Dados

http://www.asimovinstitute.org/neural-network-zoo/

- Redes Multilayer Perceptron (RMP)
- Redes Neurais Convolucionais (RNC)
- Redes Neurais Recorrentes (RNR)
- Long Short-Term Memory (LSTM)
- Redes de Hopfield
- Máquinas de Boltzmann
- Deep Belief Network
- Deep Auto-Encoders
- Generative Adversarial Network
- Deep Neural Network Capsules.
- Outros modelos ...

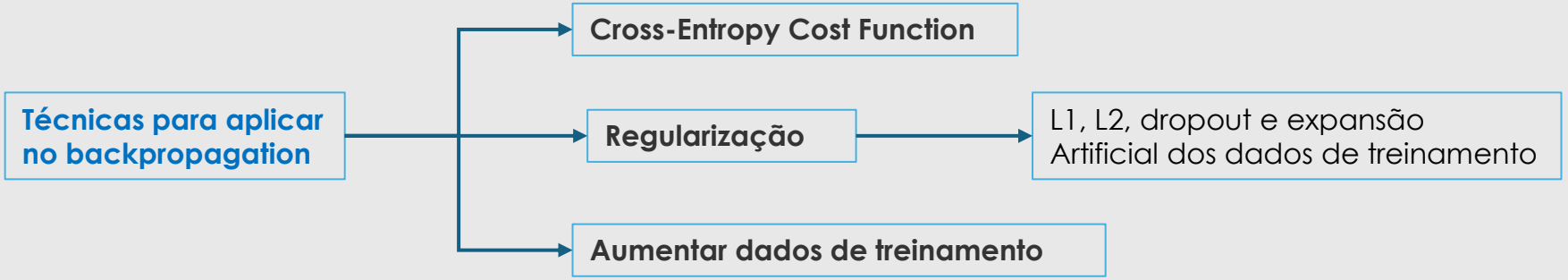
Objetivos



1986

Aumenta muito a velocidade dos cálculos pois encontra pesos e bias melhores.

Ajuda a reduzir o overfitting



Existem outras técnicas de regularização.

Técnicas de Regularização - a regularização pode ser vista como uma forma de se comprometer entre encontrar **pequenos pesos** e minimizar a função de custo original.

L2 - weight decay – adicionar um termo extra à função de custo.
L1 - a função de custo não regularizada, adicionando a soma dos valores absolutos dos pesos.
a Regularização L1 reduz o peso muito menos do que a Regularização L2.
Dropout - o Dropout não depende da modificação da função de custo. Em vez disso, no Dropout , modifica a própria rede.
Ele **elimina aleatoriamente (e temporariamente) alguns dos neurônios ocultos na rede**, deixando os neurônios de entrada e saída intocados.

Expansão Artificial dos dados de treinamento – É expandir artificialmente os dados de treinamento usando os mesmos dados e colocando eles em ângulos diferentes, rotacionando eles, também usando a técnica de "distorções elásticas", ruídos de fundo, acelerando a fala, etc.

Escolher pesos e bias inicial – Variáveis aleatórias Gaussianas normalizadas média 0 e desvio 1 mas usando a fórmula abaixo:

$1/\sqrt{n_{\text{in}}}$.

Onde nin é peso de entrada. Isso melhora a velocidade de aprendizado η e o desempenho final.

Escolher hiperparâmetros – Diminuir os dados e ir modificando os hiperparâmetros e vendo o resultado. Usar a taxa de aprendizado η .

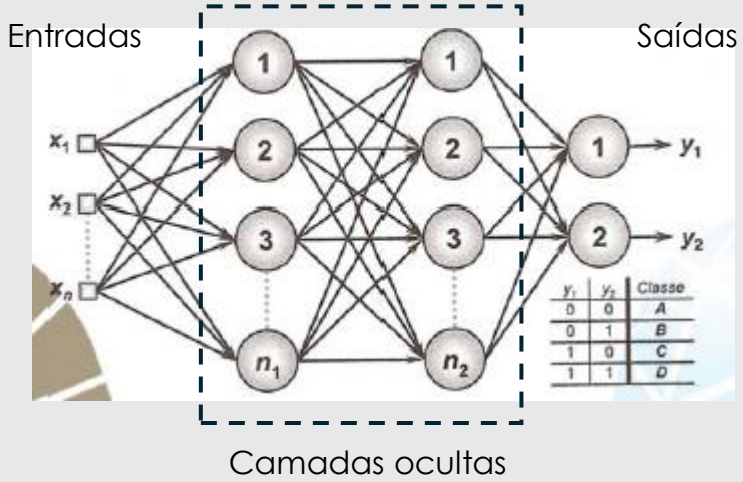
Hiperparâmetros

- Taxa de aprendizado = velocidade = η
- Número de Epoch
- Arquitetura de rede
- Mini-batch
- Batch
- Parâmetro de regularização λ ("numero de dado=n")
- Número de neurônios
- coeficiente de momentum (atrito) (μ)

Parâmetros = pesos e bias

Conclusão: Enquanto os **parâmetros** do modelo são ajustados internamente com dados durante o treinamento, os **hiperparâmetros** são definidos externamente pelo engenheiro IA e devem ser otimizados para obter o melhor desempenho do modelo.

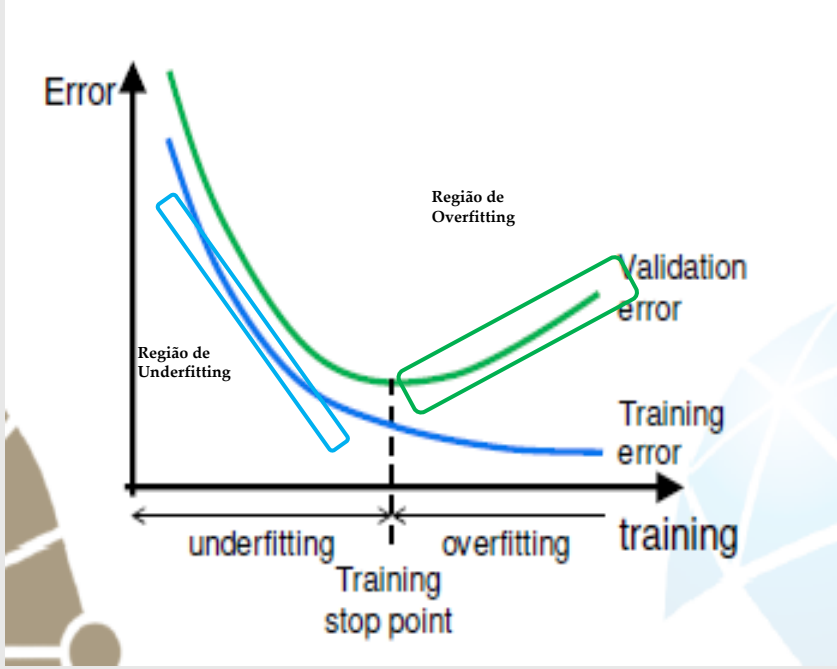
Um MLP com um número suficiente de neurônios em 2 camadas (layer) ocultas, por exemplo, podem aprender qualquer função.



OBS.: É boa prática **normalizar** ou centrar os dados no eixo zero e a quantidade de dados a ser treinado depende da quantidade de dados que existe.
Se existir 3 classes com quantidades diferentes de amostras, pegar proporcional.
Exemplo: uma classe =100 amostras, outra=300 amostras e outra =50 amostras.
Não posso pegar validação igual, por exemplo 10% de amostra, temos que pegar de cada classe, que ficaria 10-30-5, proporcional.
Muda resultados: pesos sinápticos, bias, número de neurônios, de camadas ocultas, quantidade validação, números de dados insuficientes.

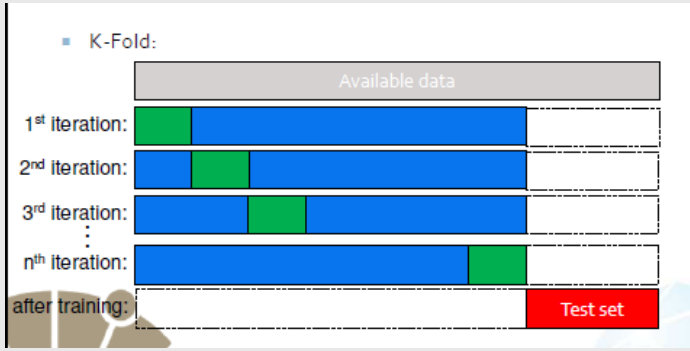
Overfitting = Especialista apenas nos dados de treinamento (erro baixo), não **generaliza** o resultado e tem alto erro de validação nos testes (curva verde).
Generalizar é a capacidade de predizer valores para dados não usados.

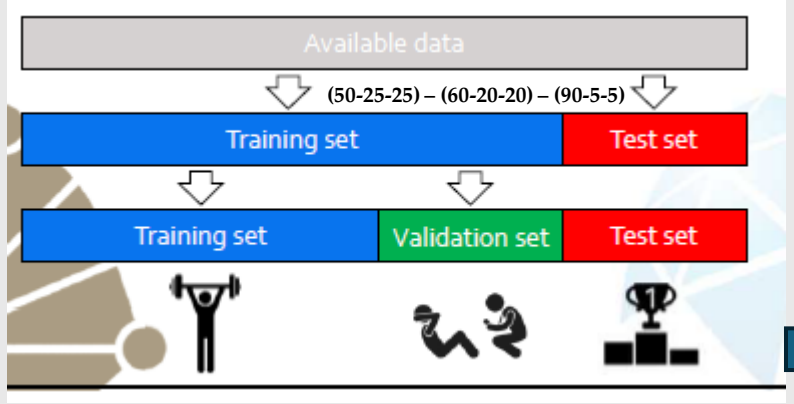
Underfitting = Não consegue representar os dados. Pode ter erros de treinamento alto (curva azul não chega próximo de zero)



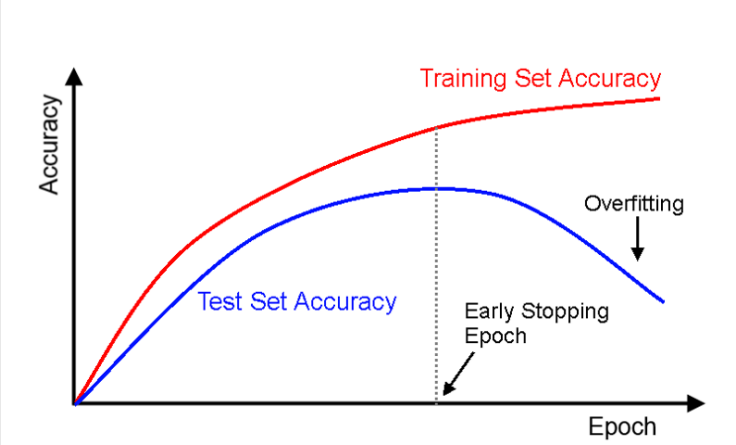
Para evitar **Overfitting** e **Underfitting** devemos parar o treinamento onde o erro pode estar diminuindo mas a validação estará piorando (data holdout=early stop) ou quando depois de um determinado epochs o erro fica constante mesmo não piorando a validação.
Por isso os dados devem ser divididos em 3 partes, **Treinamento-Validação-Testes.** Treinamento e validação ocorrem simultaneamente para controlar o aprendizado ou early stop.
Testes depois que conseguiu o **modelo final.**

Existem várias estratégias de validação, Durante o treinamento.





Modelo (py)



Stochastic Gradient Descent (SGD)

Hessian Optimization – usar a Matriz Hessiana para aproximar de um mínimo.

Momentum – modifica a descida, alterando a velocidade e colocando termo de fricção, que diminui gradualmente a velocidade, coeficiente de momentum (μ) (atrito) entre 0 e 1.

Mini-batch – Quando dados de treinamento são divididos em lotes pequenos.
É usado para diminuir o número de memória RAM a ser usada e filosofia da vetorização (Exemplo de transformar pixel de uma figura em ponto, linhas e curvas, com a possibilidade de modificar eles separadamente).
São múltiplos de 2 (64,128,256,512) . é a quantidade de dados que você deseja processar em **cada epoch**.

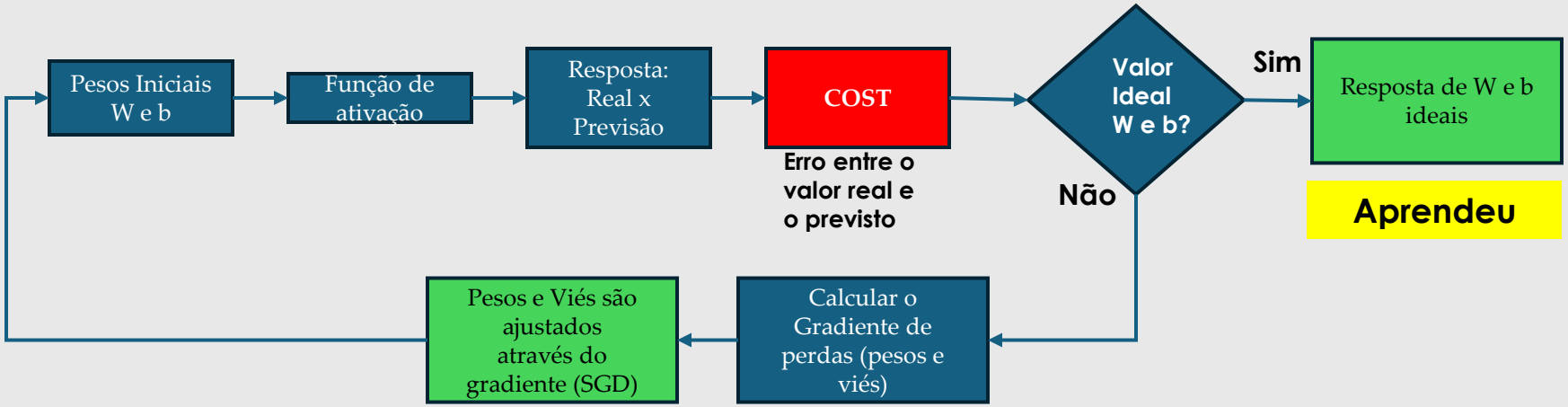
Epochs – 1 passagem completa do conjunto de dados, quando ele retorna para ajustar seus pesos sinápticos e então continua neste loop de Épocas até que o aprendizado ou o erro chegue perto de zero e não altere mais seus valores, ou pare por algum motivo definido.

Problema de Vanishing Gradient e Explosão: O desaparecimento de gradiente ocorre durante o aprendizado usando o método de gradiente e backprogation. Cada interação do treinamento os pesos das redes neurais recebem uma atualização proporcional à derivada parcial da função de cost (erro), que normalmente pode estar em cadeias.
Estes valores em cadeias são multiplicados entre si e se a derivada for muito próximas de zero, praticamente zero, os pesos iniciais podem ter uma atualização insignificante e um aprendizado lento (zero) ou houver a perda (Vanishing) do gradiente .
Quando as funções de ativação que a derivada parcial tem valores maiores, pode ocorrer o contrário, os valores irem aumentando até um valor muito grande, havendo a explosão do gradiente.
Estes problema comprometem a rede e elas se perdem ou trava.
Existem soluções para mitigar o Vanishing Gradient, como o uso de funções de ativação estáveis, técnicas de normalização e arquiteturas avançadas de redes neurais. Com essas abordagens, é possível superar esse desafio e melhorar o desempenho das redes neurais profundas em diversas aplicações de inteligência artificial, exemplos: ativação ReLU, Batch Normalization, função de otimização como o ADAM e arquitetura como a rede recorrente RNN.

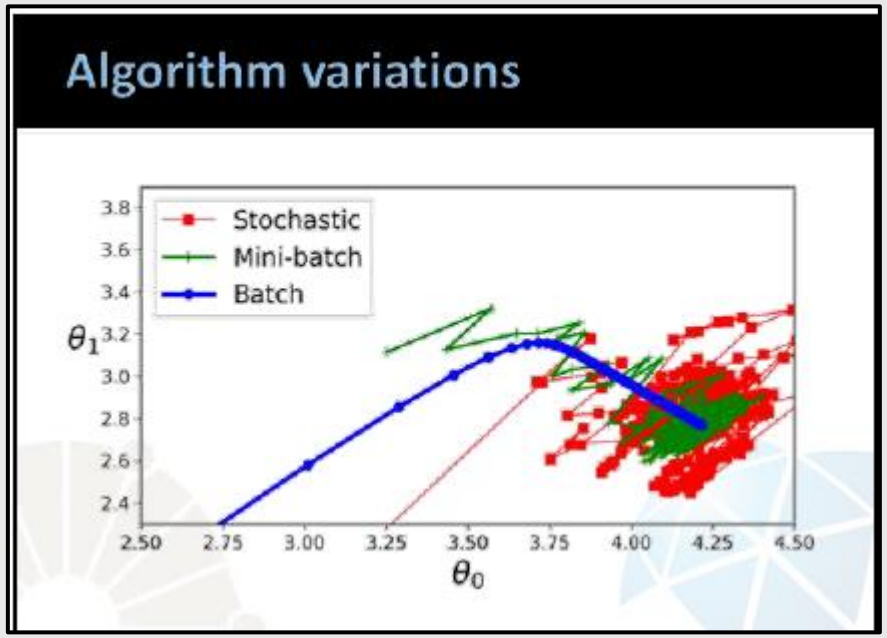
Teorema da Universalidade - As redes neurais podem computar qualquer função com apenas 2 camadas escondidas de neurônios.

A derivada parcial de **x** em relação a **y** são o Gradiente descendente que com a velocidade dos cálculos e backpropagation atua nos pesos e bias para dar uma resposta final (Objetivo)**.**

Otimizadores são vários métodos de atualização de peso e bias, para acelerar a taxa de velocidade ou de aprendizado.



Aprendendo



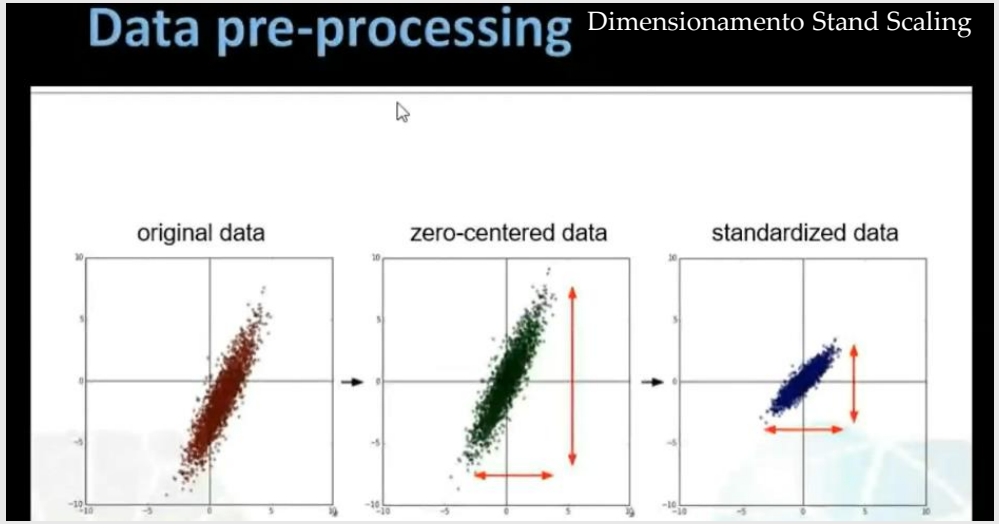
Batch – cada interação do Gradiente Descendente (**GD**) – indica tamanho do passo da derivada) usado em **todos** os exemplos/amostras de um conjunto de treinamento.
Este passo por ser muito pequeno, demora mais porém encontra os mínimos corretos (tende ao acerto).

Stochastic Gradient Descent (SGD) – Cada interação do **GD** usando **apenas 1 exemplo/amostra** de um conjunto de treinamento.
Este passo por ser muito grande é mais rápido mas pode passar pelo mínimo e não encontrar uma tendência de valor (tende ao erro).

Mini-Batch – cada interação do **GD** treinado **em n exemplos/amostras** de um conjunto de treinamento.
Este passo intermediário pode usar o melhor dos 2 métodos (Batch e Stochastic).

Preparar Dados e Métricas

A melhor estratégia depende dos dados usados e da forma trabalhada ou desejada de saída.
Antes de usar esta estratégia, os dados devem ser pré-processado, como por exemplo, centrando no eixo 0 central e padronizar, chamado Stand Scaling.



O dimensionamento de recursos é uma etapa essencial na modelagem dos algoritmos com os conjuntos de dados. Os dados que geralmente são usados para fins de modelagem são derivados através de vários meios, tais como: Questionário, Pesquisa, Investigação, etc.
Assim, os dados obtidos contêm características de várias dimensões e escalas em conjunto. Diferentes escalas dos recursos de dados afetam negativamente a modelagem de um conjunto de dados.
Isso leva a um resultado enviesado das previsões em termos de erros de classificação e taxas de precisão.
Assim, é necessário dimensionar os dados antes da modelagem.
A padronização é uma técnica de escalonamento em que torna os dados livres de escala, convertendo a distribuição estatística dos dados no formato abaixo: **média = 0 (zero) e desvio padrão = -**1
Este é o caso de aplicar a camada escondida do Batch Normalization.

Preparar Dados e Métricas – Batch Normalization

Batch Normalization é uma camada escondida que possui 4 parâmetros por entrada (γ, β, μ e σ) onde μ/σ são médias não afetadas por retropropagação (não treináveis). Na rede temos (512 Batch1 + 512 Batch2)/2 (μ/σ) = 512 parâmetros não treináveis , sendo γ o fator de escala gama e β offset beta são treináveis.
A normalização em lote é um algoritmo que torna o treinamento de redes neurais profundas mais rápido e estável. Consiste em normalizar vetores de ativação de camadas ocultas usando a média e a variância do lote atual. Essa técnica de normalização é aplicada logo antes ou depois da ativação.

Devemos lembrar que para contar os parâmetros de cada camada será multiplicar o número de entradas/ou saídas por estes 4 parâmetros de entrada, sendo a metade deles treináveis e a outra metade não treináveis.

Além de deixar mais rápido o treinamento e teste, resolve uma série de outros problemas, como por exemplo colaborar para não existir os gradientes Divergente perdidos ou gradientes explosivos.
No Keras este valor de normalização fica entre 0 e 1 mas dependendo do framework ou bibliotecas usadas, pode variar de -1 a 1.

Avaliação com Matriz de Confusão: A avaliação da matriz de confusão é muito usado depois de treinar os dados com a IA , tem as principais métricas com os resultados de saída.
Matrix de Confusão: Dela retira várias métricas e portanto é necessário compreender muito bem ela, e duas são bem usadas e temos as fórmulas abaixo, as demais métricas que vem da Matriz de confusão, pode ser usada conforme necessários:

- Precisão
- Accuracy (melhor que precisão)
- Recall
- f-score (melhor métrica)
- Sensibilidade ou Revocação
- Taxa de falsa descoberta
- Taxa de falsa emissão
- Especificidade

Usando um banco de dados público íris.csv e aplicando um programa de classificação no python, obtém um resultado que é mostrado na Matriz de Confusão que analisaremos 2 métricas e faremos a comparação.

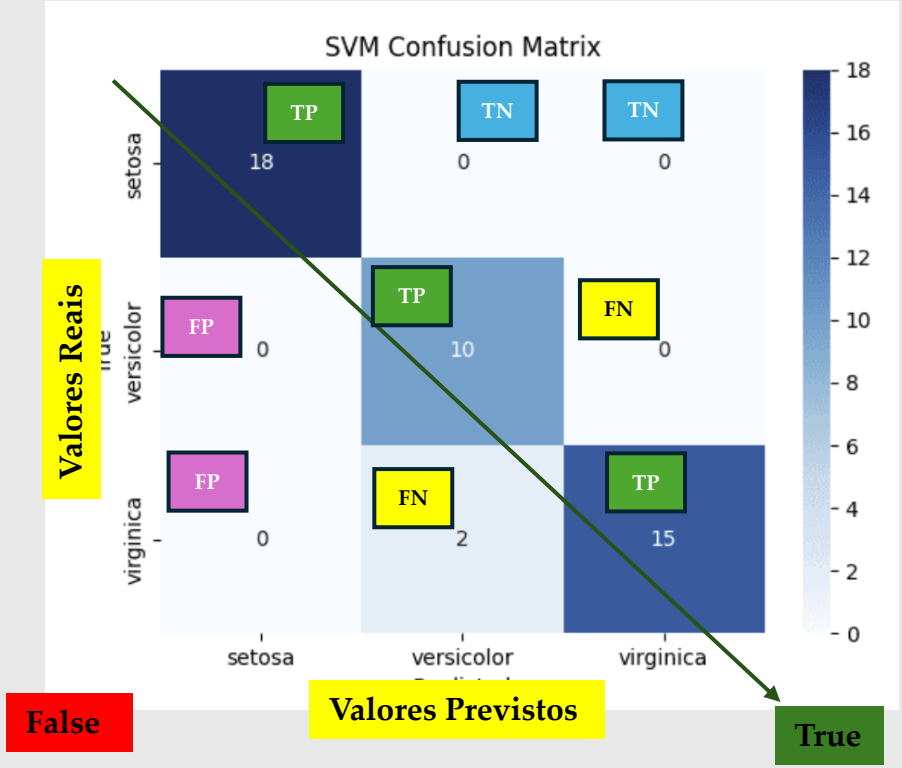
P = Precisão $P = \frac{TP}{TP+FP}$

ACC = Acurácia $Acc = \frac{TP + TN}{TP+TN+FP+FN}$

Exemplo de tabelas Simples

		1	0
Real	1	TP	FN
	0	FP	TN

Previsão



Acc setosa = 18 + 0 / (18 + 0) +(0+0) = 100%

Acc versicolor = 10+0) /(10+0)+(2 +0) = 10/12 = 83%

Acc Virginica = 15+2 / (15+0) + (2+0) = 17/ 17 = 100%

P setosa = 18/18+0 =100%
P versicolor = 10/10+2 = 83%
P virginica = 15/15+0 = 100%

Total de instância (amostra) = 45

Setosa = 18/45 = 40%
Versicolor = 10/45 = 22%
Virginica = 15/45 = 33%

O par ordenado ou **célula** da linha 3 e coluna 1 (3,1) pertence à flor setosa, par ordenado (x,y) do resultado de IA no arquivo íris.csv .
O par ordenado (2,2) é versicolor.
O par ordenado (1,3) é virginica.
Estas coordenadas das estâncias previstas (x) versus estâncias reais (y) forma a diagonal principal da matriz mostrando as classes.

Estudando o par ordenado (3,1) pertence à classe **setosa,** portanto este cruzamento é **TP**.
(2,1) e (1,1) está na coluna setosa REAL portanto mesma **classe** mas não as **instâncias** que pertencem a ela, então **FP.**
(3,2) e (3,3) As instâncias são diferentes mas referente a classe setosa, portanto **TN.**Os demais seriam **FN.**

Estudando o par ordenado (2,2) pertence à classe **versicolor,** portanto este cruzamento é **TP**.
(3,2) e (1,2) está na coluna Real versicolor portanto mesma **classe** mas não as **instâncias** que pertencem a ela, portanto **FP.**
(2,1) e (2,3) não está usando a mesma classe, portanto **TN**
Célula (1,1) e (3,3) é **FN.**

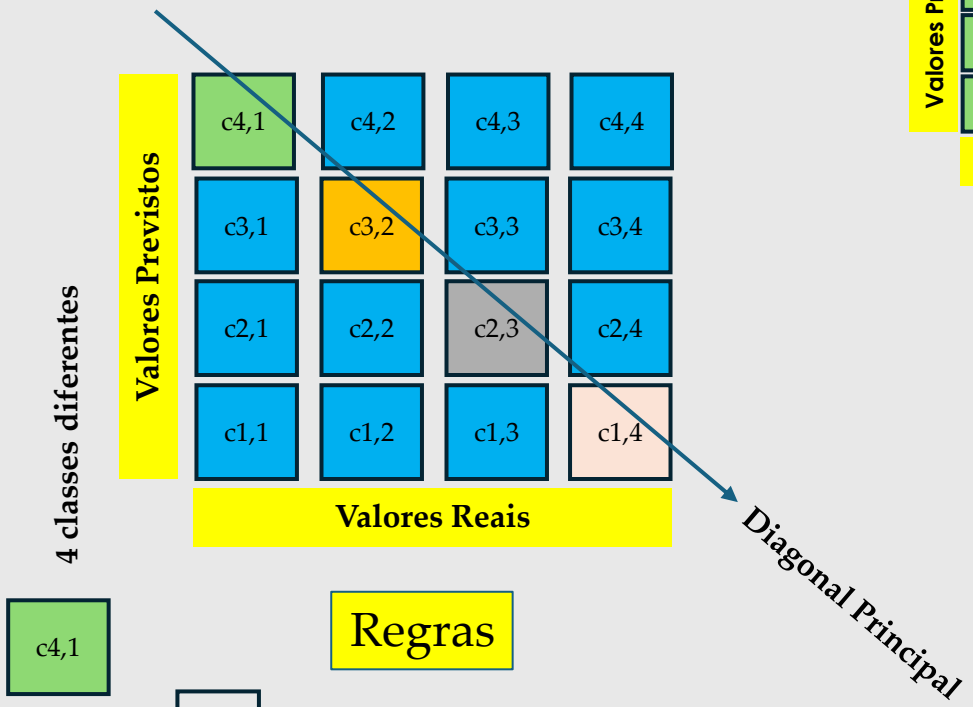
Estudando o par ordenado (1,3) pertence à classe **virginica,** portanto este cruzamento é **TP**.
(2,3) e (3,3) no eixo Real está na coluna virginica portanto mesma **classe** mas não as **instâncias** que pertencem a ela, portanto **FP.**
(1,1) e (1,2) está usando a mesma classe, portanto **TN**
As células (2,1) e (3,2) é **FN** .

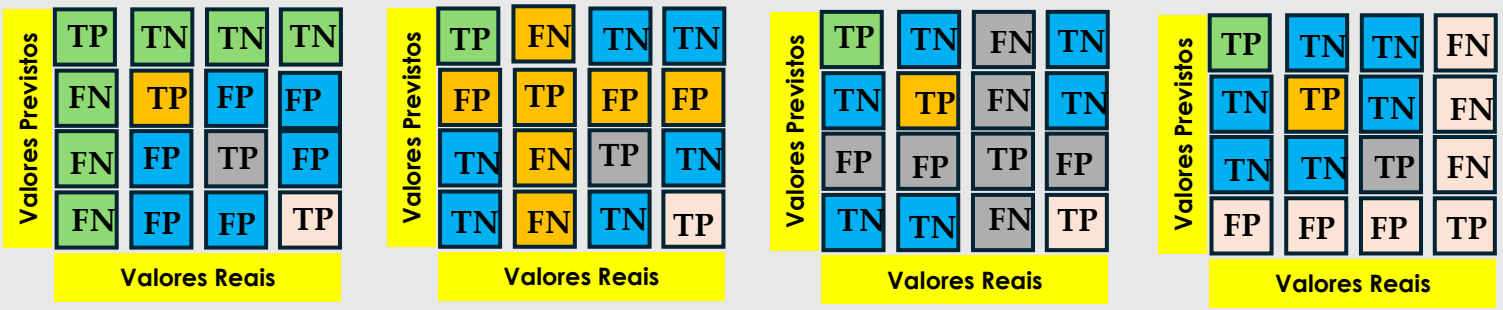
Setosa: 18 **TP** 0 **FN** 0 **FP** 0 **TN**
Versicolor: 10 **TP** 0 **FN** 2 **FP** 0 **TN**
Virginica: 15 **TP** 0 **FP** 0 **FN** 2 **TN**

ACC = Acurácia $Acc = \frac{TP + TN}{TP+TN+FP+FN}$

P = Precisão $P = \frac{TP}{TP+FP}$

Exemplo de uma matriz 4 x 4 e invertendo os eixos, no x colocado os reais e no y colocando o previsto.





Cada classe identificado todas as células os seus respectivos TP-TN-FP-FN

Regras

TP Todos que estão na diagonal principal

FP Todos que estão em classes diferentes e instâncias diferentes.

FN Todos que estão na mesma classe e no eixo dos Previstos.

TN Todos que estão na mesma classe e no eixo dos Reais

Verdadeiro Positivo (**TP**): Representa a quantidade de vezes que o modelo previu a condição como positiva e, de fato, a condição era positiva.

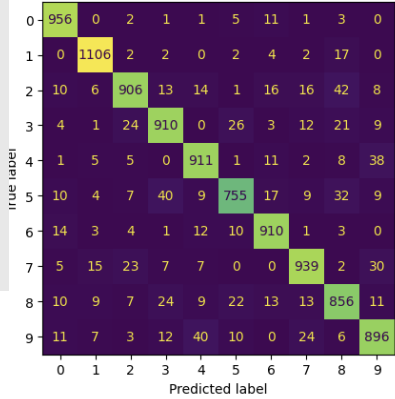
Verdadeiro Negativo (TN): Representa a quantidade de vezes que o modelo previu a condição como negativa e, de fato, a condição era negativa.

Falso Positivo (FP): Representa a quantidade de vezes que o modelo previu a condição como positiva, mas a condição era negativa.

Falso Negativo (FN): Representa a quantidade de vezes que o modelo previu a condição como negativa, mas a condição era positiva.

Exemplo 2: Um programa Python usando Keras e o banco de dados mnist apresentou a matriz de confusão e o valor de acurácia de **Test accuracy**: 0.9144999980926514, que representa 91,45%.

Após mapear a matriz de confusão e usado a fórmulas, chegamos com a acurácia de 91,69%, muito próximo da fórmula calculada pelo programa e também confirma o raciocínio correto do mapeamento.



True Label	0	1	2	3	4	5	6	7	8	9
0	TP	TN	TN	TN	TN	TN	TN	TN	TN	TN
1	FP	TP	FN	FN	FN	FN	FN	FN	FN	FN
2	FP	FN	TP	FN	FN	FN	FN	FN	FN	FN
3	FP	FN	FN	TP	FN	FN	FN	FN	FN	FN
4	FP	FN	FN	FN	TP	FN	FN	FN	FN	FN
5	FP	FN	FN	FN	FN	TP	FN	FN	FN	FN
6	FP	FN	FN	FN	FN	FN	TP	FN	FN	FN
7	FP	FN	FN	FN	FN	FN	FN	TP	FN	FN
8	FP	FN	FN	FN	FN	FN	FN	FN	TP	FN
9	FP	FN	FN	FN	FN	FN	FN	FN	FN	TP

Predicted Label

True Positive (TP): Represents the number of times the model predicted the condition as positive and, in fact, the condition was positive (main Diagonal)

True Negative (TN): Represents the number of times the model predicted the condition as negative and, in fact, the condition was negative. Everyone who is in the same classand in the axis of the Predicted

False Positive (FP): Represents the number of times the model predicted the condition as positive, but the condition was negative.Everyone who is in the same class and on the Real axis

False Negative (FN): Represents the number of times the model predicted the condition as negative, but the condition was positive.
All of which are in different classes and different instances.

TN = 24
TP = 9145
FP = 65
FN = 766

Total = 10.000

correto

ACC = Accuraci Acc = $\frac{TP + TN}{TP+TN+FP+FN}$

Acc= (9145+24) / 10000 = 0,9169
Acc = 91,69%

P = Precision P = $\frac{TP}{TP+FP}$

P= (9145) / (9145+65) = 0,9929
P = 99,29%

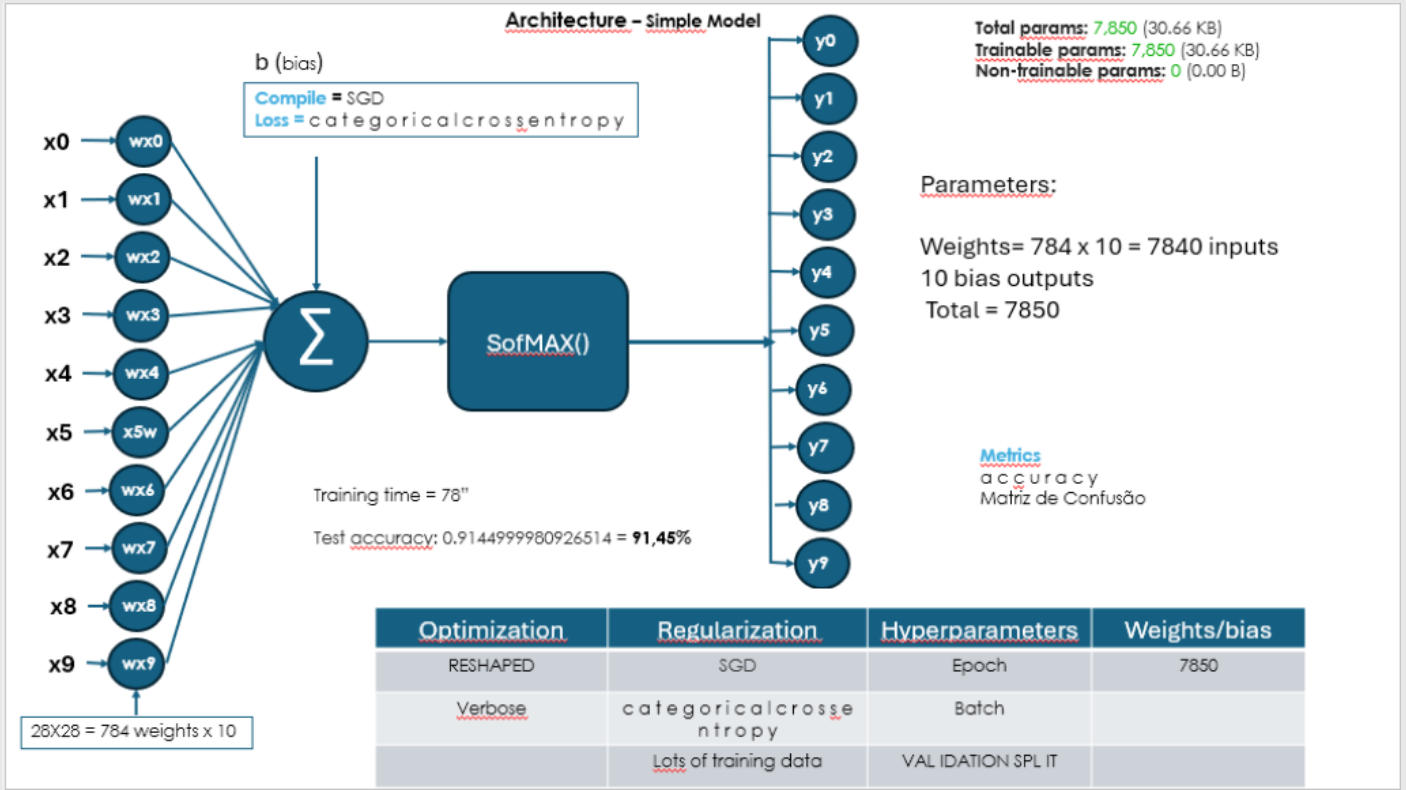
Estudar a arquitetura e parâmetros

Outra forma importante de analisar a arquitetura, hiperparâmetros e parâmetros é através do comando **`model.summary()`** do Python.
Um dos programas retornou esta resposta que é interessante a partir dela, construir a arquitetura.
Temos outro exemplo, otimizando em 4 camadas ocultas e aplicando o conceito dropout.

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
dense_layer (Dense)	(None, 10)	7,850

```
Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)
```



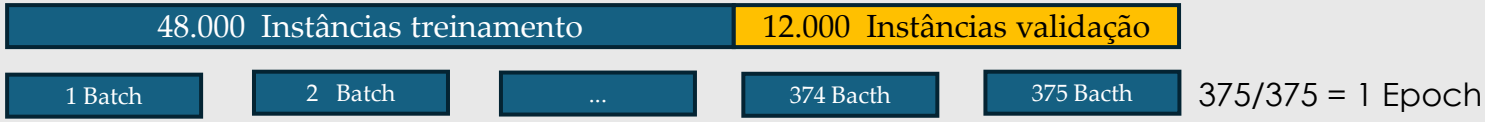
```python
# defining hyperparameters
EPOCHS = 50              # number of epochs
BATCH_SIZE = 128         # batch size
VERBOSE = 1              # printing configuration
VALIDATION_SPLIT = 0.2   # how much TRAIN is reserved for VALIDATION

# Training the model.
history = model.fit(X_train, Y_train,
  batch_size=BATCH_SIZE, epochs=EPOCHS,
  verbose=VERBOSE, validation_split=VALIDATION_SPLIT)
```

```
Epoch 1/50
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.4712 - loss: 1.8275 - val_accuracy: 0.8256 - val_loss: 0.9062
Epoch 2/50
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8200 - loss: 0.8625 - val_accuracy: 0.8567 - val_loss: 0.6592
Epoch 3/50
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8466 - loss: 0.6725 - val_accuracy: 0.8699 - val_loss: 0.5625
Epoch 4/50
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8623 - loss: 0.5779 - val_accuracy: 0.8781 - val_loss: 0.5088
Epoch 5/50
375/375 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8664 - loss: 0.5328 - val_accuracy: 0.8835 - val_loss: 0.4743
Epoch 6/50
```

Data = 60.000 instâncias.
Validation_Split=0.2 = 12000 instâncias usadas para validação.
Batch_Size = 128

1 Epoch = (60.000 – 12.000) / 128 = 375 Batchs

Na expressão "**75/375**" , o número 375 refere-se ao total de batches a ser processados durante essa época.
O número 75 indica que já foram rodados 75 batches de 375 que serão rodados para completar 1 Epoch.
O número total de batches é determinado pelo tamanho do conjunto de dados e pelo tamanho do batch (lote) definido



Epoch 12/50: 12ª epoch de um total de 50 epoch programadas.

1s: O tempo total que levou para processar os batches até o momento.

3ms/step: O tempo médio que levou para processar cada batch, que neste caso é de 3 milissegundos.

accuracy: 0.8872: Esta é a acurácia do modelo nos dados de treinamento até aquele ponto, indicando que o modelo fez previsões corretas em aproximadamente 88,72% dos casos.

loss: 0.4265: Este é o valor da perda no conjunto de dados de treinamento, refletindo o quão bem o modelo está se ajustando aos dados.

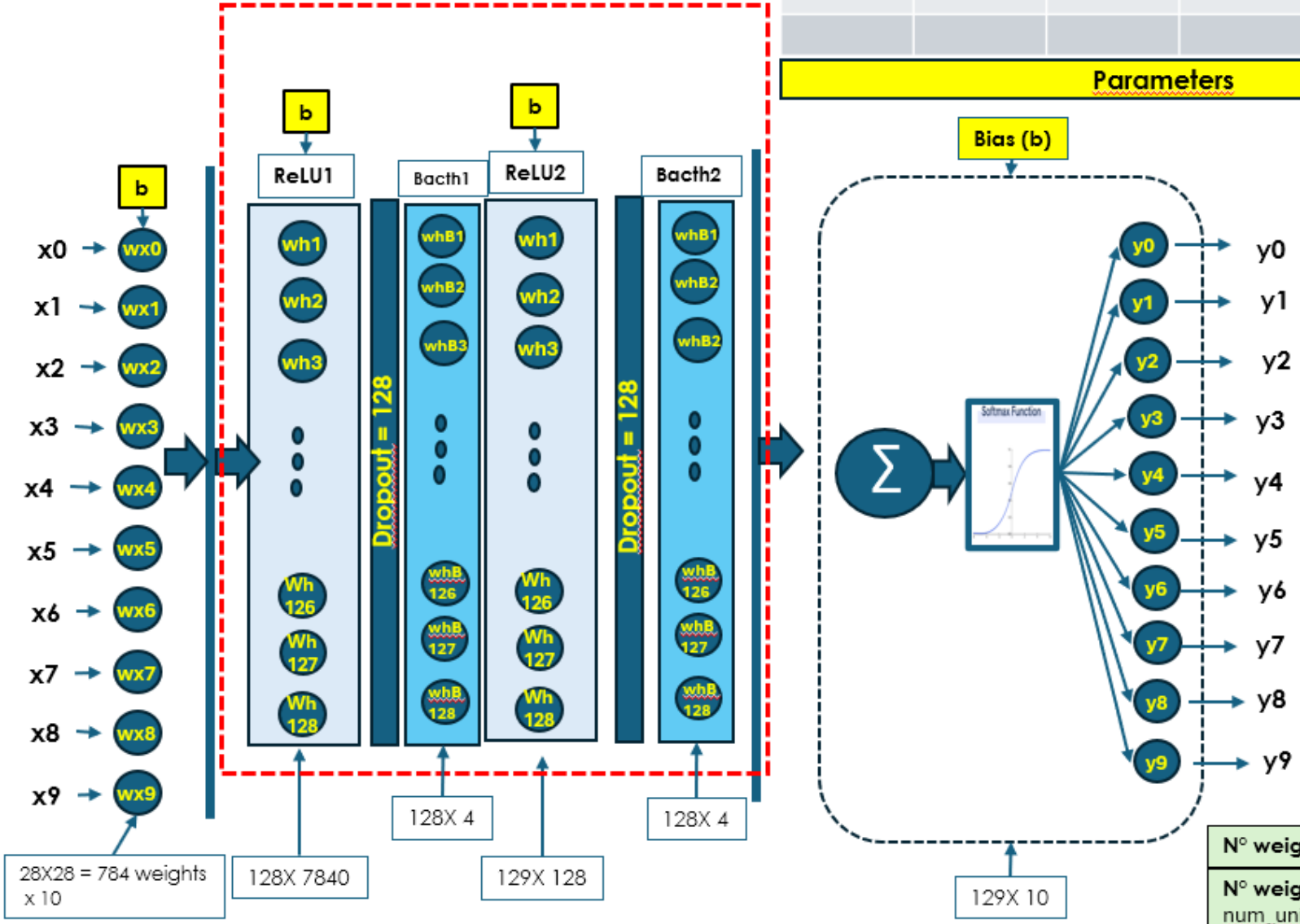
val_accuracy: 0.8986: Esta é a acurácia do modelo no conjunto de validação, que é de 89,86%. Isso mostra quão bem o modelo está generalizando para dados não vistos.

val_loss: 0.3886: Este é o valor da perda no conjunto de validação, que ajuda a monitorar o desempenho do modelo em dados que não foram usados para o treinamento.

Layer (type)	Output Shape	Param #
dense_layer (Dense)	(None, 128)	100,480
dropout_2 (Dropout)	(None, 128)	0
batch_normalization (BatchNormalization)	(None, 128)	512
dense_layer_2 (Dense)	(None, 128)	16,512
dropout_3 (Dropout)	(None, 128)	0
batch_normalization_1 (BatchNormalization)	(None, 128)	512
dense_layer_3 (Dense)	(None, 10)	1,290

Weight ReLU1	Batch1	Weight Relu2	Bacth2	Bias Output	Total	Hyperp.	Regular.	Optimiz.
100480	512	16512	512	1290	119306	Epoch	SGD	Reshap
						Batch	loss	Verbose
						VAL IDATION	data	Metrics
						N° neuros	Dropout	size-normalized
							Bacth_N	

Parameters



Training time: 111"

Test accuracy: `Test accuracy: = 0.9739000201225281`
Acc = **97,39%**

ReLU1 = (784 winput+1b)x128 neurônios)
Bacth1 = (128 x 4) =512
ReLU2 = (128 woutput+1b)x128 neurônios
Bacth2 = (512)

SoftMax = (128 woutput+1b)x10 neurônios

Bacth Normalization = It has 4 parameters per input (γ, β, μ and σ) where μ/σ are averages not affected by backpropagation (not trainable).In the network we have (512 Bacth1 + 512 Bactch2)/2 (μ/σ) = 512 non-trainable parameters.
γ - gamma scale factor and β offset beta are trainable.

```
Total params: 119,306 (466.04 KB)
Trainable params: 118,794 (464.04 KB)
Non-trainable params: 512 (2.00 KB)
```

Batch normalization is an algorithm that makes deep neural network training faster and more stable. It consists of normalizing hidden layer activation vectors using the mean and variance of the current batch. This normalization technique is applied just before or after activation.

N° weights Dense Layer = (num_units * num_inputs) + num_units

N° weights Dense activation = (num_units * num_channel input) + 1 bias per num_units

Principais Arquiteturas

As arquiteturas são criadas para resolverem ou minimizarem os grandes problemas da Inteligência Artificial. Este é o caminho que não tem fim e faz a IA avançar.
Durante a criação das arquiteturas descobrem-se otimizações da rede, que são soluções que as vezes não cria uma nova arquitetura mas resolvem ou minimizam os problemas das arquiteturas existentes.
Estas otimizações das arquiteturas são divididas em 2 grandes blocos:

a) **Regularização**: Quanto afeta o cost (perdas) e/ou gradiente divergente, que são os chamados "parâmetros" da IA, quem atua é o sistema;

b) **Hiperparâmetros**: Quando afeta a arquitetura ou um dos seus trabalhos mas a decisão de alteração ou aplicação depende do Engenheiro de IA.

As principais arquiteturas são:

Redes Neurais feedforward – caminha da esquerda para a direita

Redes Neurais recorrentes – caminha para frente e para trás (backpropagation)

Redes simétricas: São redes recorrentes mas que tem os mesmos pesos em ambas as direções.
As redes conectadas simetricamente sem unidades ocultas são chamadas de "**Redes Hopfield**". As redes conectadas simetricamente com unidades ocultas são chamadas de "**Máquinas de Boltzmann**".

Convolutional Neural Network(CNN)

Recurrent Neural Network (RNN)

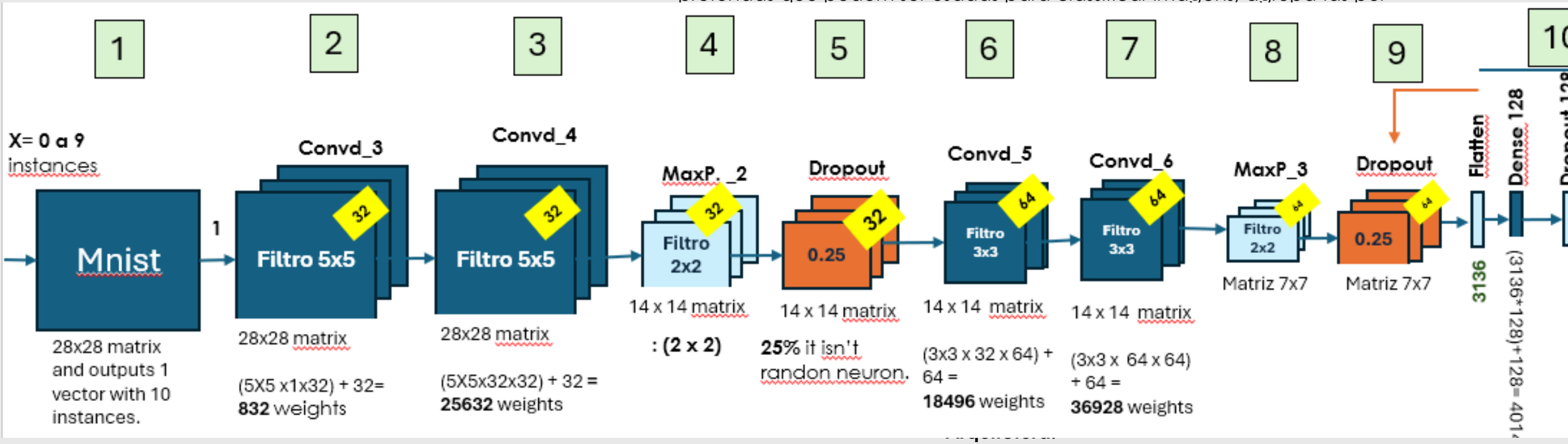
Redes Neurais Long Short Term Memory (LSTM)

Redes Neurais Gated Recurrent Unit (GRU)

GANs – Generative Adversarial Networks

Principais Arquiteturas - Redes Neurais Convolucionais (CNN)

Redes Neurais Convolucionais (ConvNets ou CNNs) são redes neurais artificiais profundas que podem ser usadas para classificar imagens, agrupá-las por



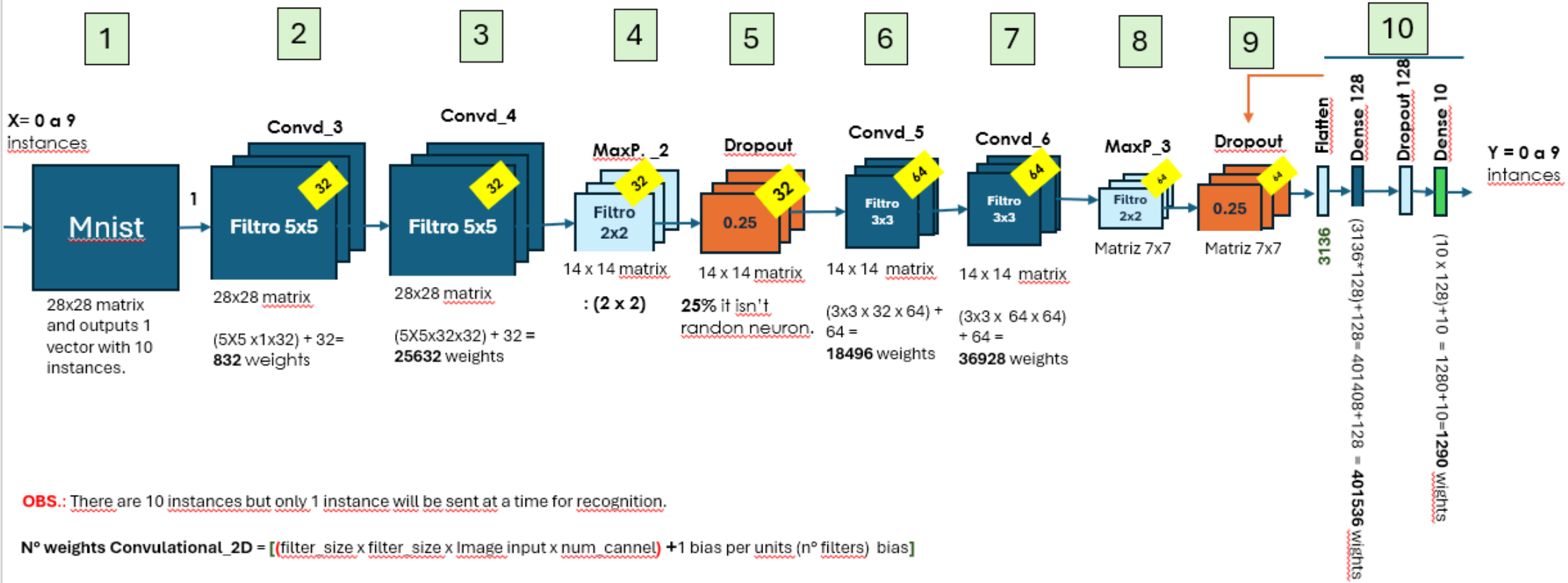

```
input_shape = (28, 28, 1)
num_classes = 10
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (5,5), padding='same', activation='relu', input_shape=input_shape),
    tf.keras.layers.Conv2D(32, (5,5), padding='same', activation='relu'),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    tf.keras.layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    tf.keras.layers.MaxPool2D(strides=(2,2)),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
model.summary()
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarni
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_6"
```

Layer (type)	Output Shape	Param #
conv2d_20 (Conv2D)	(None, 28, 28, 32)	832
conv2d_21 (Conv2D)	(None, 28, 28, 32)	25,632
max_pooling2d_12 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_6 (Dropout)	(None, 14, 14, 32)	0
conv2d_22 (Conv2D)	(None, 14, 14, 64)	18,496
conv2d_23 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_13 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_7 (Dropout)	(None, 7, 7, 64)	0
flatten_6 (Flatten)	(None, 3136)	0
dense_12 (Dense)	(None, 128)	401,536
dropout_8 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 10)	1,290

```
Total params: 484,714 (1.85 MB)
Trainable params: 484,714 (1.85 MB)
Non-trainable params: 0 (0.00 B)
```

Principais Arquiteturas - Redes Neurais Convolucionais (CNN)



OBS.: There are 10 instances but only 1 instance will be sent at a time for recognition.

N° weights Convulational_2D = [(filter_size x filter_size x Image input x num_cannel) +1 bias per units (n° filters) bias]

N° weights Dense Layer = (num_units * num_inputs) + num_units

N° weights Dense activation = (num_units * num_channel input) + 1 bias per num_units

N° Flattening

64

Entrada

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	32	33	34	35
36	37	38	39	40	41	42
43	44	45	46	47	48	49

Matrix 7x7

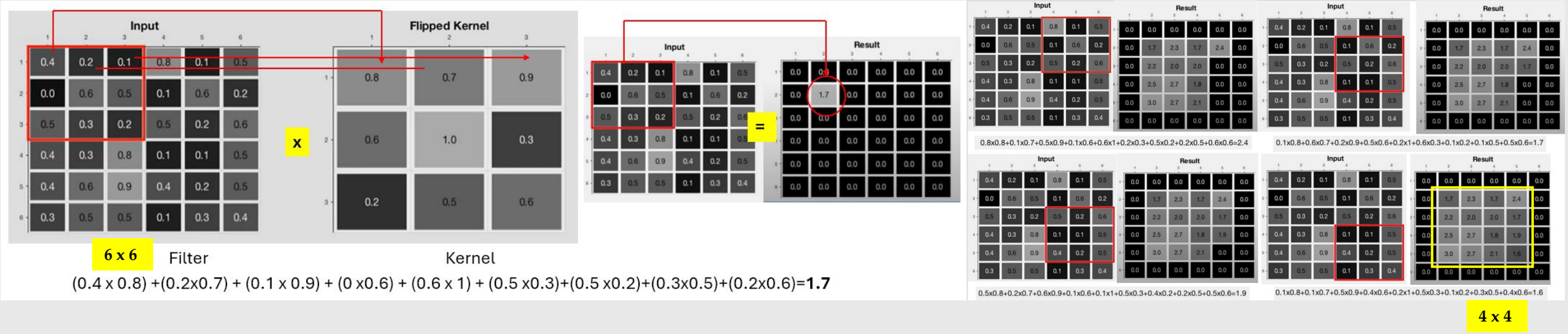
= 64 x 49 = 3136

Saída

Weight Conv_3	Weight Conv_4	Weight Conv_5	Weight Conv_6	Dense 128	Total	Hyperp.	Regular.	Optimiz.
832	25632	18496	36928	401536	484714	Epoch	SGD	reshap
						Batch	loss	Verbose
						Convolutional	Data	Metrics
						Dense Layer	Flatten (3136)	
						MAXPooling	Activation	
							Cost	
Parameters (weights + bias)						Time: 1569s		

O conjunto de dados MNIST é uma sigla que significa o conjunto de dados do Instituto Nacional de Padrões e Tecnologia Modificado. É um conjunto de dados de 60.000 pequenas imagens em tons de cinza de 28×28 pixels quadrados de dígitos manuscritos únicos entre 0 e 9. A tarefa é classificar uma determinada imagem de um dígito manuscrito em uma das 10 classes que representam valores inteiros de 0 a 9 é o objetivo desta arquitetura CNN.
Dividimos nosso infográfico em 10 partes de explicação.

Descrição1: No infográfico, o primeiro quadrado representa o banco de dados MNIST com os 10 dígitos de 0 a 9 armazenados.
Descrição 2: O segundo aplica-se a técnica de convolução da matemática para uma das 10 imagens que entrará para ser analisada. Dos 10 modelos apenas um entrará e será identificado pela IA na sua saída.
Esta convolução usa seu kernel do tamanho da própria matriz de entrada, o que significa que não reduzirá o tamanho das matrizes no final da convolução.
O que é convolução?
Veja um exemplo para entender esta técnica, utilizando um kernel do tamanho do filtro, que reduzirá o tamanho original da matriz.



Convolução: A convolução é aplicar na matriz um filtro (quadrado vermelho 3x3)do exemplo e temos uma matriz **6x6** de entrada e iremos usar uma matriz kernel para fazer a operação matemática, onde o filtro vai rodar em toda matriz. 6x6 e obter outra matriz **4x4,** com os valores de sua matriz diferente mas se for um bom kernel, com significância.
Multiplicamos cada posição da matriz dos valores de entrada pelo do kernel nas mesmas posições e somamos todo o resultado.
O primeiro dele é multiplicar (0.4 x 0.8) que são as mesmas posições. Vamos somando o resultado de todas as multiplicações de cada posição.
Quando todas as posições forem realizadas a multiplicação e somadas elas, neste exemplo o resultado foi 1.7, que é colocado na mesma posição subtraindo 1 linha e 1 coluna da matriz de entrada.
Após finalizado descemos o filtro de uma posição e realizamos as multiplicações e soma tudo de novo e encontramos um outro valor.
Fazemos este processo até que o filtro passe em todas as posições da matriz de entrada. O resultado final após cada passo aplicado finaliza o processo de convolução.

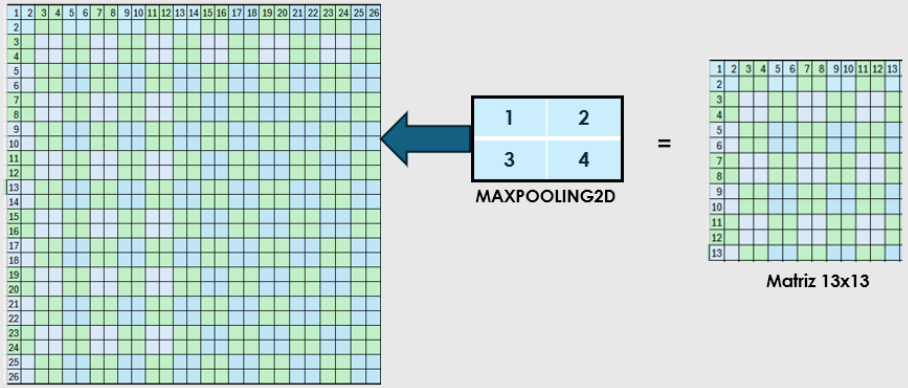
Python: Através do Python, definimos que este filtro tem o tamanho de 3x3, cuja entrada será 1 figura com a matriz de 28X28 do seu pixel, tendo um tamanho de rede neural com de 32 matrizes e portanto 32 saídas.
No programa o kernel foi definido para não alterar o tamanho da matriz, então portanto ela continuou como matriz 28X28.
Aqui existe um número de parâmetros que são ajustados pela IA e facilmente calculados pelas fórmula:

N° weights Convulational_2D = [(filter_size x filter_size x Image input x num_cannel) +1 bias per units (n° filters) bias]

Portanto a importância dos dados citados acima, que fica: **N° weights** = [(5x5) x (1) x (32)] + 32 bias = 832. Estes dados é um parâmetro da IA.

Descrição 3: Temos outra convolução igual, apenas altera o número de entradas que agora é 32. Fazendo o novo cáculo chegamos que N° weights=25632.

Descrição 4: Todos os vetores são aplicados um hiperparâmetro chamado de MaxPooling.
O MaxPooling é uma matriz 2X2 ou 3x3, trata-se de um filtro para comprimir e diminuir o tamanho de uma matriz proporcionalmente.
Este filtro é colocado na matriz de entrada que transforma 4 valores em 1. Neste caso, uma matriz 28x28 se transforma em matriz 14x14 porque estamos usando o filtro 2x2. Como esta técnica aplica-se diretamente na matriz, não existem parâmetros de IA para serem reajustados.



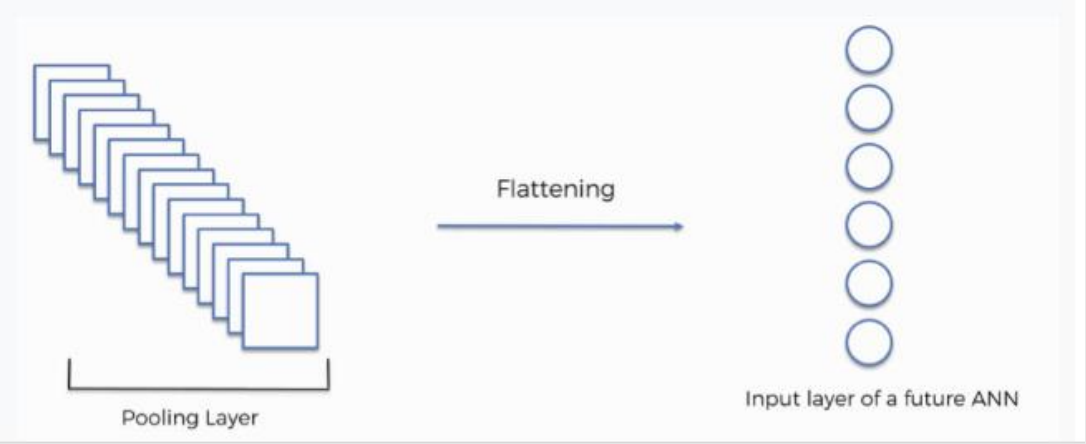
Descrição 5: Usa-se outra técnica na arquitetura, chamado de **Dropout**, onde esta técnica elimina neste caso 25% dos cálculos de neurônios das camadas escondidas. Ele não elimina nada, apenas deixa de usar em seus cálculos neurônios que são escolhidos de forma randômetra e em condições temporárias.
Ela é usada para minimizar alguns erros da rede como por exemplo o gradiente perdido e explosivo, entre outras vantagens.
Como age na arquitetura e não nos parâmetros, não existe parâmetros para serem usados.

Descrição 6: Novamente é usada a técnica de convolução mas agora o kernel tem o mesmo tamanho do filtro e com isso a matriz é reduzida em 2 colunas e 2 linhas.
N° weights = [(3x3)x(32)x(64)]+64 bias = 18496.
Descrição 7: Mesma convolução aplicada. N° weights = [(3x3)x(64)x(64)]+64 = 36928.

Descrição 8: Foi aplicado, o MaxPooling novamente, reduzindo a matriz agora para 7X7.

Descrição 9: Novamente usado a técnica de Dropout.

Descrição 10: Aqui tem a última técnica aplicada antes de rodar a rede neural.
Aplica-se a técnica chamado Flatenning que tem o objetivo de tranasformar uma matriz quadrada em apenas uma matriz de uma única coluna.
Flatenning: Como o nome desta etapa indica, vamos literalmente achatar nosso mapa de recursos agrupado em uma coluna como na imagem abaixo.



O número de entradas multiplicado pelos números de estâncias na matriz fornece a quantidade final de flattenning, ou coluna 1X n° Flatenning.
N° Flattenning = 64 x 49 = 3136. Este número ajudará a saber qual será o número de entradas do próximo bloco e depende do tamanho da matriz que chega nela, portanto não são parâmetros.
Assim que ela realizou esta técnica, passou a utilizar uma rede neural, ativando os neurônios de 128 camadas escondidas. Temos muitos parâmetros e neste caso uma nova fórmula.

N° weights Dense Layer = (num_units * num_inputs) + num_units

N° weights = (128x3136)+ 128 bias = = 401536

Usa-se novamente a técnica de Dropout, que agiliza a velocidade de processamento, por eliminar temporariamente alguns cálculos em neurônios.

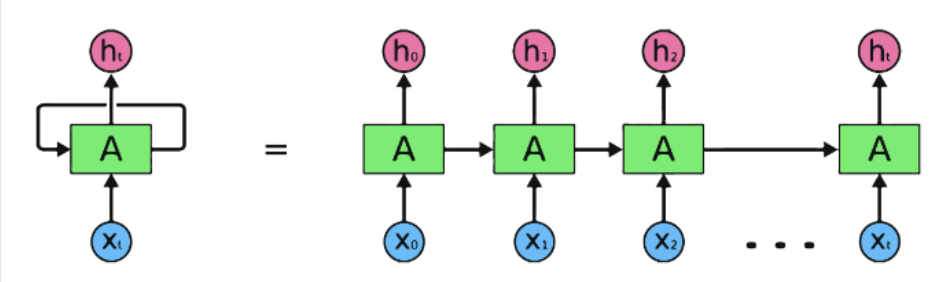
Finalmente chega na camada de neurônios de ativação, que são 10 camadas, 1 para cada dígito. O objetivo desta camada é fornecer a resposta de qual dígito de entrada que chegou, acionando a camada a qual a IA concluiu que era o dígito de entrada e bloqueando os demais 9 dígitos.

N° weights = (10 x 128)+10 bias = 1290. Portanto temos um total de weights de 484.714 as quais alguns deles aleatoriamente não entrarão nos cálculos para agilizar o processo, que neste caso com esta arquitetura, levou apenas 1569 segundos em 5 Epochs e um acerto de **96,4%.**

Principais Arquiteturas - RNN, LSTM, and GRU

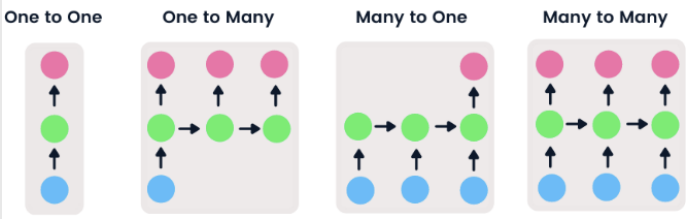
A arquitetura GRU é o resultado de duas antecessoras, a RNN e a LSTM.

As Recurrent Neural Networks (RNNs). Uma rede neural recorrente é o tipo de rede neural artificial (RNA) que usa uma memória interna para lembrar de entradas passadas. Por isso usada para prever preços de ações, gerar texto de transcrições e tradução automática.
Na rede neural tradicional, as entradas e as saídas são independentes umas das outras, enquanto a saída em RNN depende de elementos anteriores dentro da sequência.
As redes recorrentes também compartilham parâmetros em cada camada da rede. Em redes feedforward, existem pesos diferentes em cada nó. Enquanto o RNN compartilha os mesmos pesos dentro de cada camada da rede e durante o gradiente descendente, os pesos e a base são ajustados individualmente para reduzir a perda.



Tutorial de Rede Neural Recorrente (RNN) | DataCamp

Na figura acima, a entrada Xt conterá um valor de entrada que ao passar pelo neurônio A se tornará passado e será usado para prever o próximo número na sequência. No RNN, a informação percorre o loop, de modo que a saída é determinada pela entrada atual e pelas entradas recebidas anteriormente.
A camada de entrada X processa a entrada inicial e a passa para a camada intermediária A. A camada intermediária consiste em várias camadas ocultas, cada uma com suas funções de ativação, pesos e vieses. Esses parâmetros são padronizados na camada oculta para que, em vez de criar várias camadas ocultas, ele crie uma e faça um loop.
Em vez de usar a **retropropagação tradicional**, as redes neurais recorrentes usam algoritmos de retropropagação ao longo do tempo (BPTT) para determinar o gradiente. Na retropropagação, o modelo ajusta o parâmetro calculando erros da saída para a camada de entrada. O BPTT soma o erro em cada etapa de tempo, pois o RNN compartilha parâmetros em cada camada.
As redes **feedforward** têm entrada e saída únicas, enquanto as redes neurais recorrentes são flexíveis, pois o comprimento das entradas e saídas pode ser alterado.
Existem diferentes arquiteturas como na figura abaixo:



Principais Arquiteturas - RNN, LSTM, and GRU

As redes binárias simples são boas em classificação binária mas não com imagens por exemplo.
Então surgiu redes como rede neural convolucional CNN que é uma rede neural feed-forward capaz de processar dados espaciais como imagens.
As redes RNN vieram para cobrir as tarefas de séries temporais e dados sequenciais.
Ao treinar um modelo feed-forward, a CNN usa uma retropropagação simples e a RNN usa a retropropagação ao longo do tempo para calcular a perda.
A RNN não pode ter restrição no comprimento das entradas e saídas e RNN trabalha em loops para lidar com dados sequenciais.
Modelos RNN simples geralmente enfrentam dois problemas principais. Esses problemas estão relacionados ao gradiente.

1. **Gradiente de desaparecimento (Vanish)** - ocorre quando o gradiente se torna tão pequeno que a atualização dos parâmetros se torna insignificante, eventualmente, o algoritmo para de aprender.

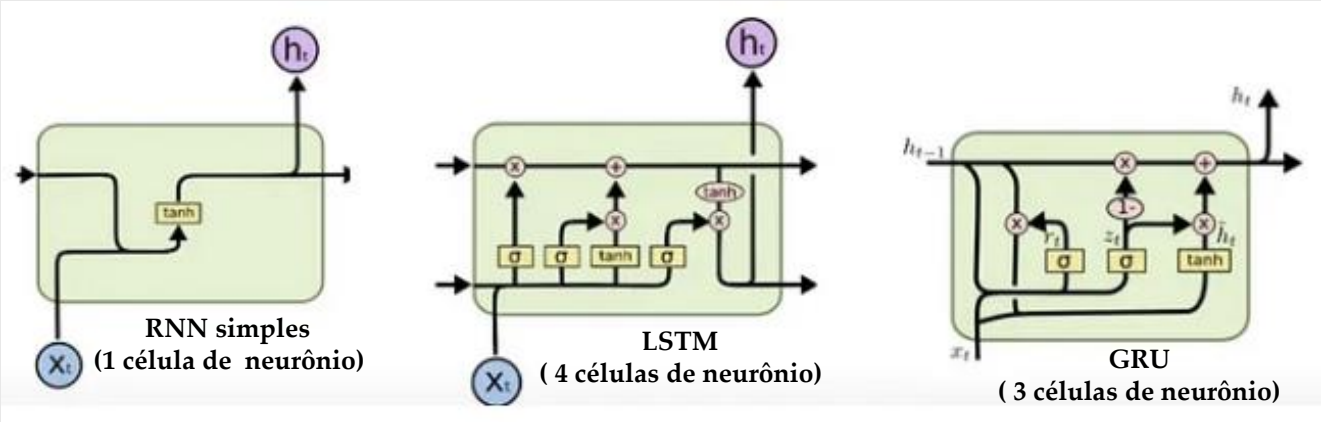
2. **Gradiente explosivo** - ocorre quando o gradiente se torna muito grande, o que torna o modelo instável. Nesse caso, gradientes de erro maiores se acumulam e os pesos do modelo se tornam muito grandes. Esse problema pode causar tempos de treinamento mais longos e baixo desempenho do modelo.

As redes simples também tem memórias curtas e estes problemas podem ser resolvidos com redes avançadas como **LSTM** e **GRU**.
A Memória de Longo e Curto Prazo (**LSTM**) é o tipo avançado de RNN, que foi projetado para evitar problemas de gradiente de desaparecimento e explosão.

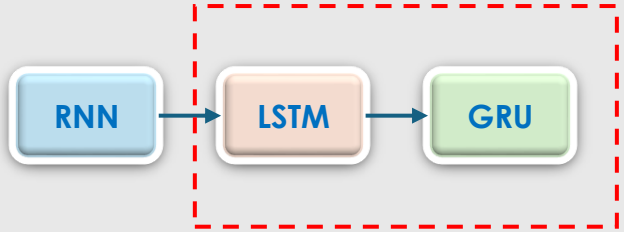
O LSTM possui módulos repetitivos como a RNN, mas a estrutura é diferente. Em vez de ter uma única camada de ativação tanh, o LSTM tem **quatro** camadas interativas que se comunicam entre si. Essa estrutura de quatro camadas ajuda o LSTM a reter a memória de longo prazo.

A unidade recorrente fechada (**GRU**) é uma variação do LSTM, pois ambos têm semelhanças de design e, em alguns casos, produzem resultados semelhantes. O GRU usa uma porta de **atualização** e uma porta de **reinicialização** para resolver o problema do **gradiente de fuga**. Essas portas decidem quais informações são importantes e as passam para a saída. Os portões podem ser treinados para armazenar informações de muito tempo atrás, sem desaparecer com o tempo ou remover informações irrelevantes.
Nahead_array determina a quantidade de previsão de saída que terá no tempo, por exemplo, 1 dia depois 1 saída, ou 8 dias depois, 8 saídas.

Um problema que o GRU resolve em relação ao LSTM é sua arquitetura mais simples, que leva a tempos de treinamento mais rápidos e requer menos parâmetros para aprender. Os GRUs conseguem isso combinando as portas de esquecimento e entrada em uma única porta de atualização, simplificando o mecanismo de bloqueio em comparação com o LSTM.
O GRU tem 1 célula de ativação com tangente hiperbólico (**Tanh**) e 2 células sigmoide (**σ**), diminuindo uma célula sigma em relação ao LSTM.
LSTM: As redes LSTM normalmente têm mais parâmetros devido às portas adicionais e ao estado da célula.



Desbloqueando o poder da unidade recorrente fechada (GRU): entendendo a arquitetura inovadora | por Sachinsoni | Média (medium.com)



LSTM: Os LSTMs são mais adequados para tarefas que exigem controle explícito sobre dependências de longo prazo, como modelagem de linguagem ou tradução automática.

GRU: Os GRUs são frequentemente preferidos em cenários em que os recursos computacionais são limitados ou onde tempos de treinamento mais rápidos são desejados, como em aplicativos em tempo real ou ao trabalhar com conjuntos de dados menores.

Exemplos de arquitetura dos 3 modelos após o comando summary() de Python.

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 20)	440
dropout (Dropout)	(None, 20, 20)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 20)	820
simple_rnn_2 (SimpleRNN)	(None, 20, 20)	820
simple_rnn_3 (SimpleRNN)	(None, 20)	820
dense (Dense)	(None, 1)	21

Total params: 5,844 (22.83 KB)
Trainable params: 2,921 (11.41 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,923 (11.42 KB)

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 50)	10,400
lstm_1 (LSTM)	(None, 50)	20,200
dense_1 (Dense)	(None, 25)	1,275
dense_2 (Dense)	(None, 1)	26

Total params: 95,705 (373.85 KB)
Trainable params: 31,901 (124.61 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 63,804 (249.24 KB)

Model: "sequential_11"

Layer (type)	Output Shape	Param #
gru_1 (GRU)	(None, 50, 50)	7,950
gru_2 (GRU)	(None, 50)	15,300
dense_21 (Dense)	(None, 25)	1,275
dense_22 (Dense)	(None, 1)	26

Total params: 73,655 (287.72 KB)
Trainable params: 24,551 (95.90 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 49,104 (191.82 KB)

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 50)	10,400
lstm_1 (LSTM)	(None, 50)	20,200
dense_1 (Dense)	(None, 25)	1,275
dense_2 (Dense)	(None, 1)	26

Total params: 95,705 (373.85 KB)
Trainable params: 31,901 (124.61 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 63,804 (249.24 KB)

LSTM Params= 4(1+50+1)50 = 10.400

LSTM Params= 4(50+50+1)50 = 20.200

Params= 50 x 25 + 25 bias = 1275

Params= 25 x 1 + 1 bias = 26

LSTM weights=4(input+layer+1 bias) layer

Model: "sequential_11"

Layer (type)	Output Shape	Param #
gru_1 (GRU)	(None, 50, 50)	7,950
gru_2 (GRU)	(None, 50)	15,300
dense_21 (Dense)	(None, 25)	1,275
dense_22 (Dense)	(None, 1)	26

Total params: 73,655 (287.72 KB)
Trainable params: 24,551 (95.90 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 49,104 (191.82 KB)

GRU Params=3×(nentrada+nsaída+2)×nsaída

- nentrada é a dimensão da entrada (50, neste caso).
- nsaída é o número de unidades ocultas (também 50).
- 2 número de bias para cada porta.

GRU Params= 3x(1+50+2) x50

GRU Params= 3x(50+50+2)x50

Params = 50 x 25 + 25 bias = 1275

Params= 25 x 1 + 1 bias = 26

LSTM weights=3(input+layer+1 bias) layer

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 20)	440
dropout (Dropout)	(None, 20, 20)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 20)	820
simple_rnn_2 (SimpleRNN)	(None, 20, 20)	820
simple_rnn_3 (SimpleRNN)	(None, 20)	820
dense (Dense)	(None, 1)	21

Total Parameters=(input_features×units)+(units×units)+units

RNN Params= (1x20) + (20x20)+ 20

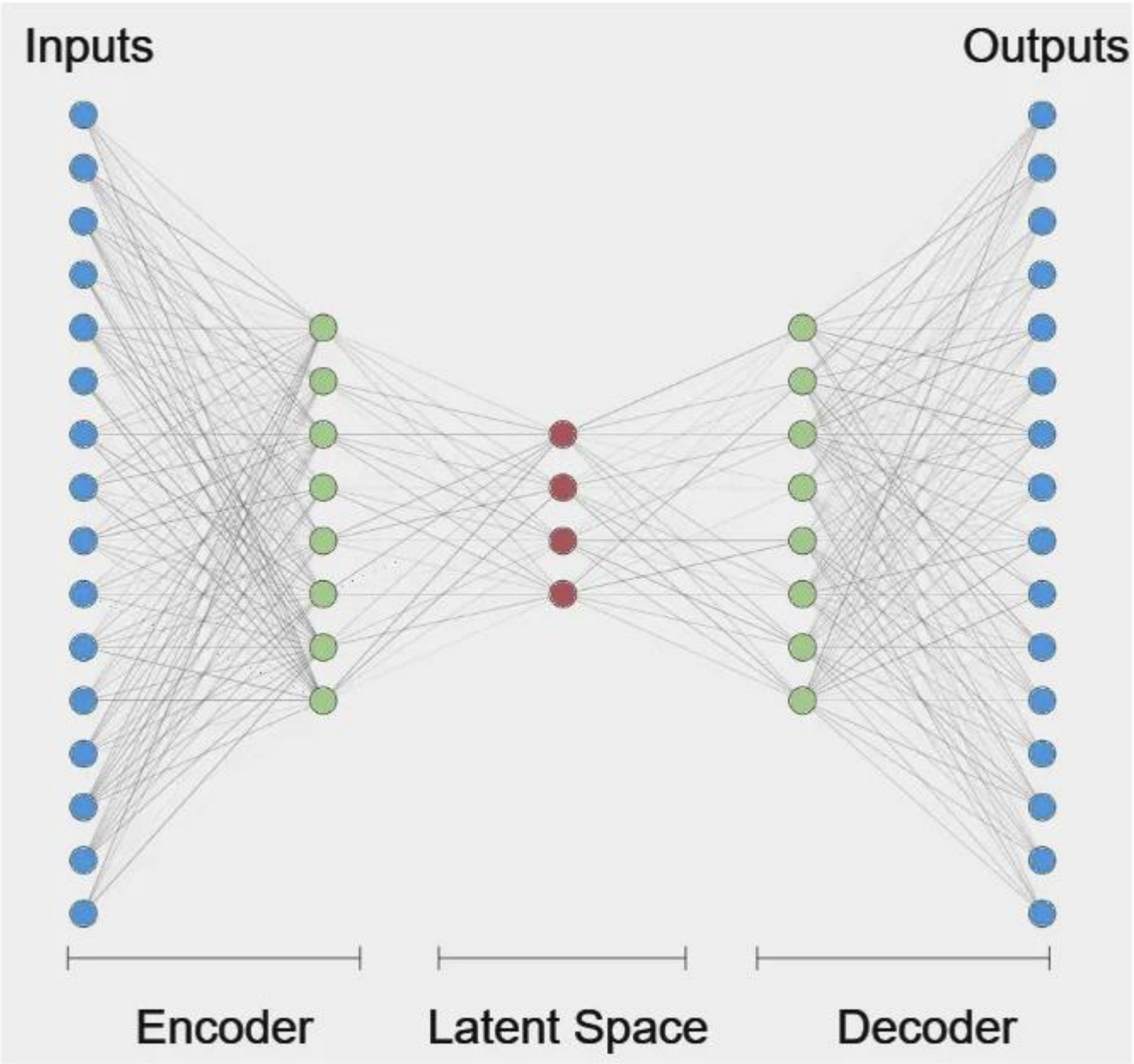
RNN Params= (20x20)+(20x20)+20

RNN Params= (20x20)+(20x20)+20

RNN Params= (20x20)+(20x20)+20

Params= 20 x 1 + 1 bias = 21

Principais Arquiteturas – Auto Encoder (AE)



Autoencoder. Gerado a partir de NN SVG (alexlenail.me)

Encoder(Codificador): Representa a parte inicial do *autoencoder* que recebe os dados de entrada da rede e tem como objetivo de sintetizar as principais informações dos dados de entrada e transforma-los para um espaço latente.

Latent Space(Espaço latente): Representa o resultado do encoder, no caso espaço latente representa um espaço de menor dimensionalidade que o original, isto é, uma representação dos dados de forma mais compacta.

Decoder(Decodificador): Para redução de dimensionalidade tem como objetivo reconstruir os dados de entrada a partir do espaço latente gerado pelo encoder. Já no caso da remoção de ruído.

Aplicações do Autoencoder:

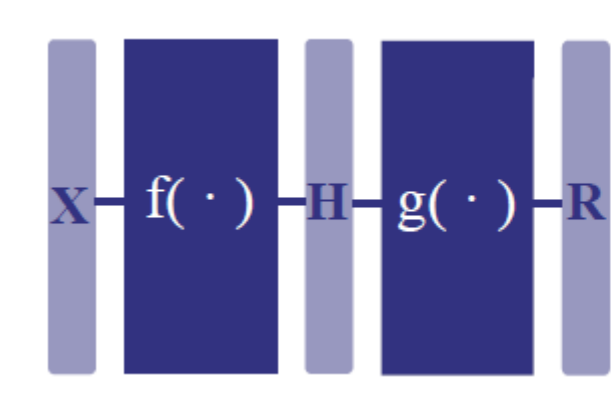
Redução de Dimensionalidade: o *autoencoder* pode ser utilizado para lidar com o problema da "maldição da dimensionalidade", ou seja, quando os inputs apresentam uma grande dimensionalidade(tamanho). E isso representa um problema, pois uma alta dimensionalidade requer um grande tempo e poder de processamento, além de dificultar a convergência dos algoritmos. Então, o *autoencoder* pode ser utilizado para aprender uma boa representação compacta dos dados originais(inclusive dados com relações não lineares) removendo informações irrelevantes ou redundantes, portanto fornece uma dimensionalidade muito inferior, mas com uma alta capacidade de representação.

Remoção de Ruído: podemos fazer um autoencoder que remove o ruído dos dados. Um caso bem comum é a remoção de ruído de imagens.

Detecção de Anomalias: após o treinamento em dados normais relevantes para o problema em análise, espera-se que o *autoencoder* tenha a capacidade de reconstruir eficazmente esses casos padrões. No entanto, quando se deparar com um exemplo atípico ou anômalo, é provável que o *autoencoder* enfrente desafios significativos na reconstrução, resultando em uma notável diferença em relação aos dados normais. Com isso, é possível identificar as anomalias, isto é, casos atípicos.

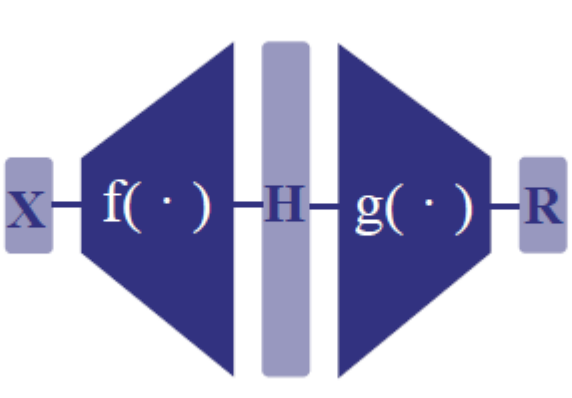
O autoencoder tem simétrica em relação ao espaço latente e tem que gerar um espaço latente bem representativo para que o decoder possa reconstruir fielmente os dados.
O objetivo final dos autocodificadores é extrair informações úteis, isso pode ser aplicado ao aprendizado de recursos ou à separação de ruído-sinal e um valor baixo de perda não implica aprendizado.
A maneira como podemos forçar o aprendizado é adicionando algumas restrições. A entrada não é perfeitamente reconstruída. Essas restrições são inseridas por meio de termos de regularização que dependem de H. O mecanismo de restrição depende da aplicação.

AE Caso Trivial



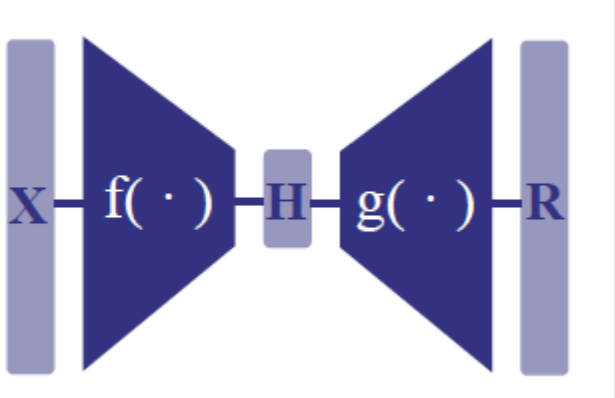
H=f(X)=X
R=g(H)=X
Loss é 0

AE Caso Overcomplete



X C H
R=g(H)=X
Loss é 0

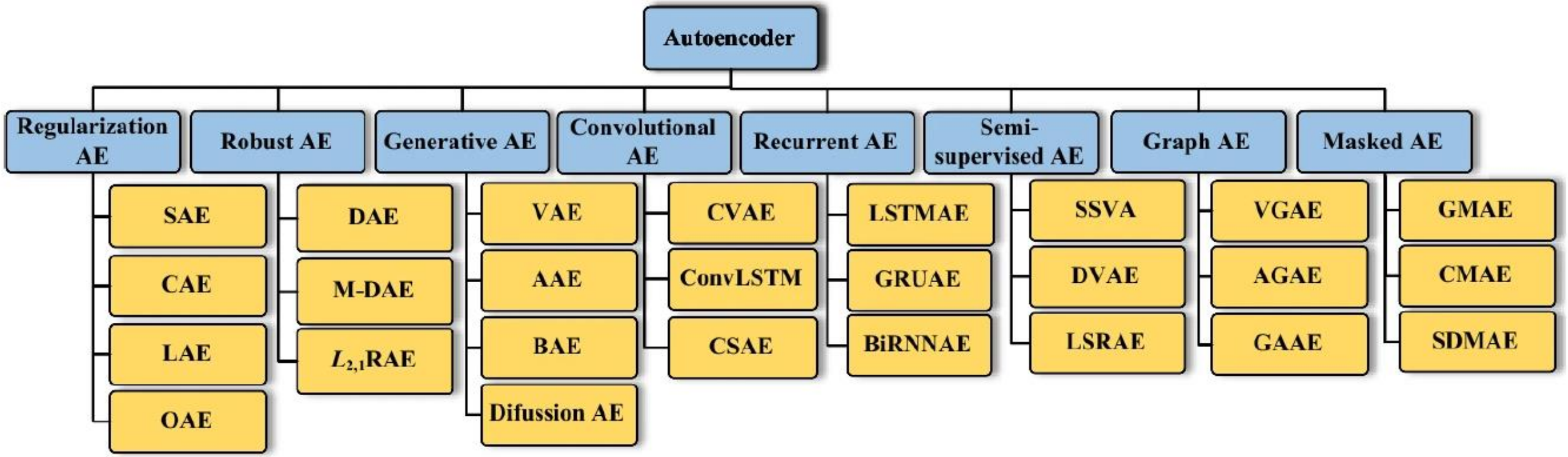
AE Caso Undercomplete



Se dermos alta potênciaa f e g, chegaremos a:
$g(.)=f(.)^{-1}$
Loss é 0

Tipos de Auto Encoder (AE)

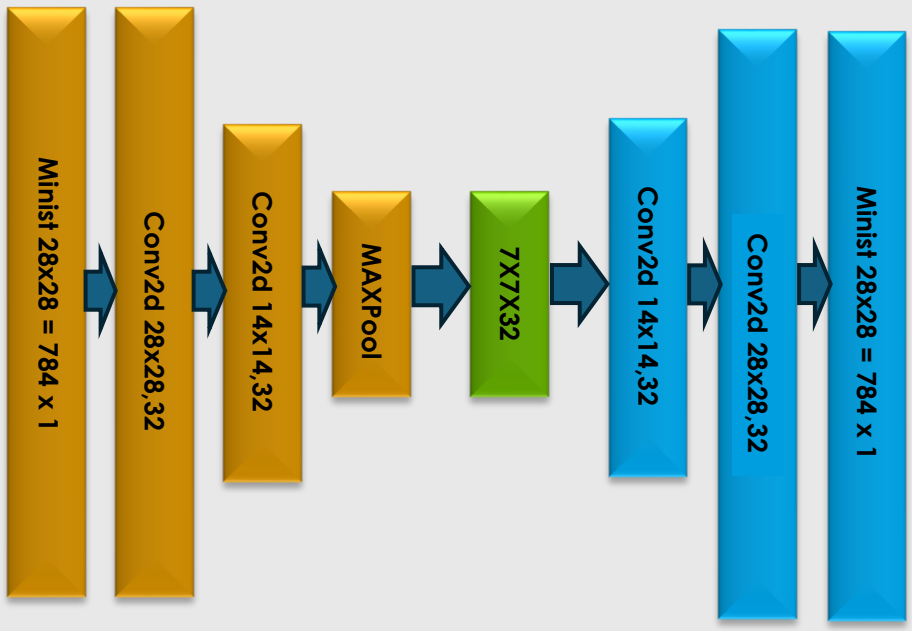
No entanto, essa taxonomia confunde aplicação e arquitetura.



Berahmand, K., Daneshfar, F., Salehi, E. S., Li, Y., & Xu, Y. (2024). Autoencoders and their applications in machine learning: a survey. Artificial Intelligence Review, 57(2), 28.

Aprendizagem representacionale aprendizagem de características: Redução da dimensionalidade, códigos e compressão.
Anomalia: Automação/controle, medicina e telecomunicações.
Redução de ruído e melhoria de resolução: Diminuição de ruídos e melhoria da resolução.

autoencoder convolucional Keras



= Encoder = Latent Space = Decoder

1 - O dataset **MNIST** está sendo **carregado** e preprocessado para normalização (dividido por 255 para colocar os valores dos pixels entre 0 e 1) e para garantir que as imagens tenham formato (28, 28, 1).
2 - reprocess: Normaliza e reorganiza o shape do array de entrada.
3 - noise: Adiciona ruído aleatório às imagens com um fator de ruído de 0.2
display isplay_latent: Funções para visualizar as imagens, tanto as originais quanto as reconstruídas, bem como o vetor latente gerado pelo encoder.
4 - **Encoder**: Usa várias camadas convolucionais (com Conv2D) e pooling (MaxPooling2D) para reduzir as dimensões das imagens. Neste caso, a imagem de entrada (28, 28, 1) vai para a forma (7, 7, 32) depois das operações de convolução e pooling.
5 - **Decoder**: Reconstroi a imagem original a partir do vetor latente, usando camadas de transposição de convolução (Conv2DTranspose) e, por fim, uma convolução com uma função de ativação sigmoid para garantir que a saída esteja entre 0 e 1.
6 - Modelo final (Autoencoder): O **autoencoder** é definido usando o encoder e decoder. Ele foi compilado com o otimizador **Adam** e a função de perda **binary_crossentropy**.

Encoder: Convolução 2D com 32 filtros e kernel de 3x3, com ativação ReLU e padding "same", Max Pooling com uma janela de 2x2, Convolução 2D com 32 filtros e kernel de 3x3, com ativação ReLU e padding "same", Max Pooling com uma janela de 2x2.
Decoder: Convolução 2D Transposta com 32 filtros e kernel de 3x3, com strides 2 e ativação ReLU, Convolução 2D Transposta com 32 filtros e kernel de 3x3, com strides 2 e ativação ReLU,Convolução 2D final com 1 filtro, kernel de 3x3 e ativação sigmoid.

```
from keras.datasets import mnist
from keras.models import Model
def preprocess(array):
    """Normalizes the supplied array and reshapes it."""
    array = array.astype("float32") / 255.0
     array = np.reshape(array, (len(array), 28, 28, 1))
    return array
def noise(array):
     """Adds random noise to each image in the supplied array."""
     noise_factor = 0.2 ## New value
          noisy_array = array + noise_factor * np.random.normal(
        loc=0.0, scale=1.0, size=array.shape
    )
    return np.clip(noisy_array, 0.0, 1.0)
def display(array1, array2):
      """Displays ten random images from each array."""
    n = 10
         indices = np.random.randint(len(array1), size=n)
    images1 = array1[indices, :]
    images2 = array2[indices, :]
    plt.figure(figsize=(20, 4))
                for i, (image1, image2) in enumerate(zip(images1, images2)):
        ax = plt.subplot(2, n, i + 1)
        plt.imshow(image1.reshape(28, 28))
        plt.gray()
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)
        ax = plt.subplot(2, n, i + 1 + n)
        plt.imshow(image2.reshape(28, 28))
        plt.gray()
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)
    plt.show()
```

```
def display_latent(array):
  latent_vector_aux = encoder.predict(array)
  m, n, p, q = np.shape(latent_vector_aux)
  plt.figure(figsize=(20, 4))
  plt.imshow(array[0,:,:,0])
  plt.axis('off')
  plt.figure(figsize=(20, 4))
  for aux in range(q):
    plt.subplot(1, p, aux+1)
    plt.imshow(latent_vector_aux[0,:,:,aux])
    plt.axis('off')
  output = decoder.predict(latent_vector_aux)
  plt.figure(figsize=(20, 4))
  plt.imshow(output[0,:,:,0])
  plt.axis('off')
(train_data, _), (test_data, _) = mnist.load_data()
train_data = preprocess(train_data)
test_data = preprocess(test_data)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input
encoder = Sequential([
                layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2), padding="same"),
                layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2), padding="same"),
    ])
decoder = Sequential([
                layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same"),
                layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same"),
                layers.Conv2D(1, (3, 3), activation="sigmoid", padding="same"),
    ])
img = Input(shape = (28, 28, 1))
latent_vector = encoder(img)
output = decoder(latent_vector)
autoencoder = Model(inputs = img, outputs = output)
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
EPOCHS = 10
BATCH_SIZE = 128
history = autoencoder.fit(
    x=train_data,
    y=train_data,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    shuffle=True,
    validation_data=(test_data, test_data),
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 134s 281ms/step - loss: 0.2452 - val_loss: 0.0730
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 137s 272ms/step - loss: 0.0724 - val_loss: 0.0694
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 273ms/step - loss: 0.0696 - val_loss: 0.0679
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 271ms/step - loss: 0.0683 - val_loss: 0.0673
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 272ms/step - loss: 0.0675 - val_loss: 0.0666
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 271ms/step - loss: 0.0668 - val_loss: 0.0663
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 272ms/step - loss: 0.0665 - val_loss: 0.0658
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 127s 271ms/step - loss: 0.0661 - val_loss: 0.0654
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 127s 271ms/step - loss: 0.0657 - val_loss: 0.0651
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 127s 272ms/step - loss: 0.0655 - val_loss: 0.0648
```

```python
# Compile the autoencoder
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")

# Fitting
EPOCHS = 10
BATCH_SIZE = 128
history = autoencoder.fit(
    x=train_data,
    y=train_data,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    shuffle=True,
    validation_data=(test_data, test_data),
)
```

Validation Data: Durante o treinamento de um modelo, o conjunto de dados de validação é usado para avaliar o desempenho do modelo em dados que ele ainda não viu. Isso ajuda a monitorar o treinamento e a evitar o overfitting. Neste caso estamos usando o mesmo conjunto de teste.
Este método somente é bom, para a arquitetura de autoencoder.

469/469 = 1 Epoch

60.000 Instâncias de treinamento

1 Batch | 2 Batch | ... | 468 Bacth | 469 Bacth

Data = 60.000 instâncias.
Batch_Size = 128

1 Epoch = (60.000) / 128 = 468,75 Batchs

Loss = 0.2451 até 0.0655
Val_loss = 0.0730 até 0.0648

23 minutos

Noise=from 0,4 to 0.2

Data = 60.000 instâncias.
Batch_Size = 128

1 Epoch = (60.000) / 128 = 468,75 Batchs

Loss = 0.2495 até 0.0653
Val_loss = 0.0729 até 0.0647

30 minutos

Conclusão: Diminuiu o ruído em 50% e somente o tempo de compilação aumentou, os resultados foram os mesmos.

```
encoder.summary()
decoder.summary()
autoencoder.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D) 1	(None, 28, 28, 32) 2	3 320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0

Total params: 9,568 (37.38 KB)
Trainable params: 9,568 (37.38 KB)
Non-trainable params: 0 (0.00 B)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 32)	9,248
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	9,248
conv2d_2 (Conv2D)	(None, 28, 28, 1)	289

Total params: 18,785 (73.38 KB)
Trainable params: 18,785 (73.38 KB)
Non-trainable params: 0 (0.00 B)

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
sequential (Sequential)	(None, 7, 7, 32)	9,568
sequential_1 (Sequential)	(None, 28, 28, 1)	18,785

Total params: 85,061 (332.27 KB)
Trainable params: 28,353 (110.75 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 56,708 (221.52 KB)

1 2D convolutional layer (CNN – applies filters to extract features from images).

2 Layer output: batch size (none), image height and width dimension 28x28, 32 is the number of filters or channels applied by the layers.

3 Params= 320
filter Conv2D = 3x3= 9 weights therefore 9x32=288 weights
bias = 1 per layer = 32 Therefore 32+288 = 320

Params= 9248
Params= (filterxqtd_filterxqtd_input) =(9x32x32)w+32b=9248

Params= 9248
Params= (filterxqtd_filterxqtd_input) =(9x32x1)w+1b=289

= 9248+320

= 9248+9248+289 = 18785

```
encoder.summary()
decoder.summary()
autoencoder.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 64)	640
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0

Total params: 37,568 (146.75 KB)
Trainable params: 37,568 (146.75 KB)
Non-trainable params: 0 (0.00 B)

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_transpose_2 (Conv2DTranspose)	(None, 14, 14, 64)	36,928
conv2d_transpose_3 (Conv2DTranspose)	(None, 28, 28, 64)	36,928
conv2d_5 (Conv2D)	(None, 28, 28, 1)	577

Total params: 74,433 (290.75 KB)
Trainable params: 74,433 (290.75 KB)
Non-trainable params: 0 (0.00 B)

Model: "functional_5"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 28, 28, 1)	0
sequential_2 (Sequential)	(None, 7, 7, 64)	37,568
sequential_3 (Sequential)	(None, 28, 28, 1)	74,433

Total params: 336,005 (1.28 MB)
Trainable params: 112,001 (437.50 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 224,004 (875.02 KB)

Layer: Aumentando o layer de 32 para 64 e compilando novamente o mesmo exercício original autoencoder, faremos uma conclusão.

60.000 Instâncias de treinamento

1 Batch | 2 Batch | ... | 468 Bacth | 469 Bacth

Data = 60.000 instâncias.
Batch_Size = 128

1 Epoch = (60.000) / 128 = 468,75 Batchs

Loss = start 0.2451 até 0.0655
Val_loss = start 0.0730 até 0.0648

23 minutos

Noise=from 0,4 layer = 64,2

Data = 60.000 instâncias.
Batch_Size = 128

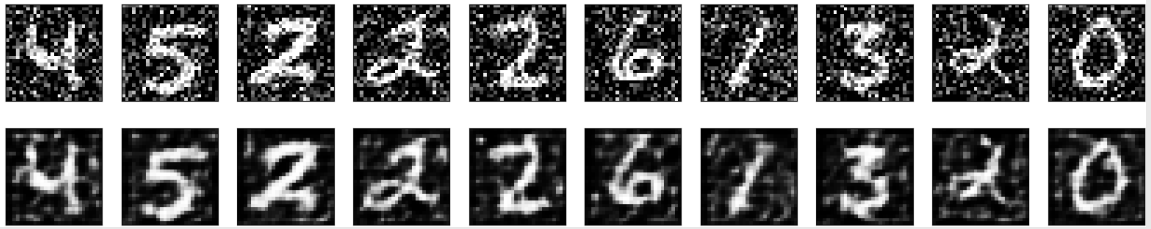
1 Epoch = (60.000) / 128 = 468,75 Batchs

Loss = start 0,2117 to 0,0630
Val_loss = start 0,0697 to 0,0625

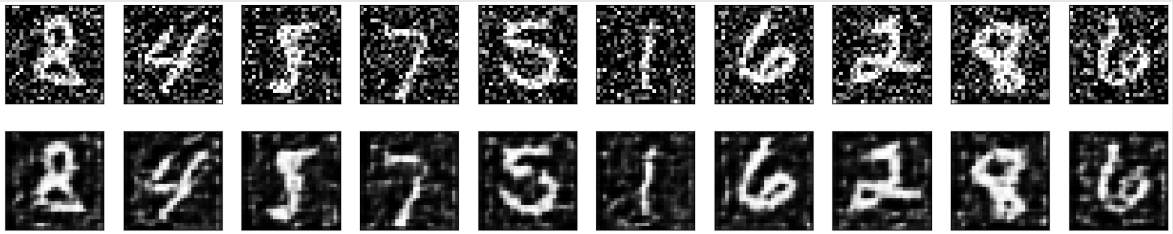
60 minutos

Conclusão: Os parâmetros aumentaram muito, cerca de 3,95 vezes os parâmetros originais. O tempo foi de 1 hora mas os resultados de loss e val_loss mudaram pouco e nas imagens geradas não houve melhoras.

Originais: Layer=32



Novo: Layers=64



Redes Generativas Adversariais (GAN)

Nas redes de IA mais comuns: Temos as entradas X que são escritas em funções e apresentam a saída Y. O sistema de IA busca imitar o comportamento de um sistema real, tanto a entrada como a saída, fazendo inferências e tomar decisões dependendo da capacidade de generalizar novos dados entre a entrada e saída do sistema.

Nas redes Generativas: São ferramentas que criam novos dados, simula cenários e obtêm insights mais profundos sobre os dados. Para isso ela não usa a entrada real mas utiliza de "ruídos" para imitar o comportamento "real" de entrada baseados nas suas saídas.

Um dos mais populares são a GAN para imagens: 1 – **Gerador** cria uma nova imagem, 2 – **Discriminador** tenta classificar como falsa ou verdadeiro, 3 – **Convolução** ajuda o Discriminador a extrair características importantes da imagem, os pesos das redes são ajustados com base no erro do discriminador e 4 – **Camada de Atenção**, processo se repete, com o gerador tentando enganar o discriminador e o discriminador ficando cada vez melhor em distinguir e classificar as imagens.

Uma visão geral das 3 abordagens de sistemas generativos:

1 – Modelos Generativos Autoregressivos: Geram os elementos sequencialmente, com cada elemento sendo condicionado ao elemento anterior.

2 – Autoencoder Varicional: VAE são modelos consistem em duas partes: um encoder que comprime os dados em um espaço latente e um decoder que tenta reconstruir os dados originais a partir dessa representação comprimida.

3 – Rede Generativa Adversariais: GANs envolvem duas redes neurais: um gerador que cria novos dados e um discriminador que tenta distinguir entre os dados reais e os gerados. Essas duas redes competem entre si, levando a uma melhoria contínua na qualidade dos dados gerados

Existem outros modelos que surgem das combinações destes 3 sistemas principais.

Estudando um exemplo:

Conclusão: Nós tivemos 10 aulas mostrando todos os conceitos de Deep Learning para a engenharia, algumas aulas para treinar os conceitos e buscar as soluções para os exercícios, compreendendo cada etapa do processo e um projeto final que envolve os conceitos aprendidos.

O projeto final adotado foi dividido em 3 estruturas, a qual chamamos de **Taxonomia da Disciplina DLE.**

Bibliografia:

1 - Elements Of AI
University of Helsinki

2 – Deep Learning Book (100 capítulos**)**
Capítulo 1 - Uma Breve História das Redes Neurais Artificiais - Deep Learning Book

3 – Material UNESP - Aprendizado Supervisionado de Maquina (SML)
Dr. Alexandre da Silva Simões e Dr. Rodrigo Capobianco Guido

4 – ChatGPT
OpenAI

5 – Gemini
Google

6 - Wikipedia